

Capítulo 13 — Observabilidad, Evaluación y Optimización

Sección 01: Introducción a la observabilidad en sistemas de IA

Los capítulos anteriores de este módulo construyeron sistemas de IA cada vez más complejos: agentes con memoria persistente, pipelines de recuperación de conocimiento, coordinación entre múltiples modelos, planificación y razonamiento en cadena. Todos esos sistemas se diseñaron y probaron. Pero ninguno de ellos se midió de forma sistemática. La pregunta que este capítulo responde es: ¿cómo saber si esos sistemas siguen funcionando correctamente una semana, un mes o un año después de que se desplegaron?

La respuesta es la observabilidad. Y en sistemas de IA, la observabilidad es un problema cualitativamente más difícil que en el software tradicional.

Por qué los sistemas de IA son diferentes

En el software tradicional, el comportamiento incorrecto es casi siempre binario: una función lanza una excepción, un servicio devuelve un código de error, una base de datos rechaza una consulta. Los errores son ruidosos y detectables. Un sistema bien monitoreado alerta cuando algo falla.

Los sistemas de IA fallan de una manera diferente. Pueden responder con HTTP 200, con latencia aceptable, con tokens dentro del presupuesto y, al mismo tiempo, estar produciendo respuestas incorrectas, irrelevantes o desactualizadas. El sistema funciona en términos técnicos y falla en términos de valor. Esta forma de falla silenciosa es lo que hace que la observabilidad de IA sea un dominio propio, no una extensión del monitoreo de software convencional.

Un asistente de atención al cliente puede responder con fluidez durante semanas mientras cita precios desactualizados. Un sistema de RAG puede recuperar documentos y construir respuestas coherentes mientras utiliza fragmentos que quedaron obsoletos hace tres meses.

Un agente puede completar sus ciclos de planificación y acción mientras toma decisiones subóptimas que ninguna métrica de infraestructura captura.

Para detectar estos problemas se necesitan instrumentos que observen no solo si el sistema responde, sino qué responde, con qué información lo hace, con qué calidad y con qué tendencia en el tiempo.

Las cuatro dimensiones de la observabilidad para sistemas de IA

La observabilidad de un sistema de IA no es un número ni una métrica. Es un conjunto de perspectivas que, tomadas juntas, permiten entender qué está ocurriendo realmente dentro del sistema.

Primera dimensión: observabilidad de la inferencia. Qué recursos consume el sistema al operar: cuánto tiempo tarda en responder, cuántos tokens usa por solicitud, cuánto cuesta cada llamada al modelo, con qué frecuencia alcanza los límites de la API, qué porcentaje de las solicitudes falla por errores técnicos. Esta dimensión es la más cercana al monitoreo tradicional de software y la más fácil de instrumentar. Sin embargo, por sí sola solo dice si el sistema está vivo y cuánto cuesta. No dice si es útil.

Segunda dimensión: observabilidad del contexto. Qué información exacta recibió el modelo en cada solicitud: qué documentos recuperó el sistema RAG y de qué fuentes, qué versión del system prompt estaba activa, qué historial de conversación se incluyó, qué herramientas ejecutó el agente y con qué resultados. Esta dimensión responde la pregunta que hace el ingeniero cuando el sistema produce una respuesta inesperada: ¿qué tenía el modelo disponible en ese momento?

Tercera dimensión: observabilidad de la calidad. Qué tan buenas son las respuestas que el sistema produce: son relevantes para la consulta del usuario, son factualmente correctas respecto al conocimiento disponible, son coherentes con el tono y las restricciones definidas, satisfacen al usuario que las recibió. Esta dimensión requiere técnicas de evaluación propias —tanto automáticas como humanas— que el capítulo desarrolla en las secciones siguientes.

Cuarta dimensión: observabilidad del comportamiento. Qué patrones de uso exhibe el sistema en producción: qué tipos de consultas recibe con más frecuencia, qué temas generan más sesiones de seguimiento, cuándo los usuarios reformulan su pregunta (señal de que la primera respuesta no satisfizo), cuáles son los momentos del día con mayor demanda, qué consultas el sistema no logra responder bien de manera consistente.

Cada dimensión requiere instrumentación diferente. El AI Engineer que solo implementa la primera dimensión tiene un sistema monitoreado pero no observable. La observabilidad completa requiere las cuatro.

El problema que la observabilidad resuelve

Considérese un sistema de soporte al cliente basado en RAG que lleva seis meses en producción. El equipo técnico reporta que el sistema tiene 99.5% de disponibilidad, latencia promedio de 800ms y un costo por consulta dentro del presupuesto. El equipo de negocio reporta que la satisfacción de los usuarios cayó 15 puntos en los últimos dos meses y que el volumen de escalaciones a agentes humanos aumentó un 30%.

¿Qué está fallando?

Sin observabilidad del contexto, el equipo no puede saber si el sistema está recuperando documentos irrelevantes. Sin observabilidad de la calidad, no puede saber si las respuestas son factualmente incorrectas. Sin observabilidad del comportamiento, no puede identificar qué tipos de consultas están causando el problema.

El equipo que tiene las cuatro dimensiones instrumentadas puede responder esa pregunta en horas. El equipo que solo tiene la primera dimensión puede tardar semanas buscando el problema en el lugar incorrecto.

La estructura del capítulo

El capítulo está organizado para construir una capacidad de observabilidad completa, desde los fundamentos conceptuales hasta la operación práctica.

Bloque de métricas y evaluación (secciones 01 a 03): los fundamentos de la observabilidad en sistemas de IA, las métricas de calidad y desempeño, y las técnicas de evaluación tanto automática como humana.

Bloque de instrumentación (secciones 04 y 05): la trazabilidad de prompts y contexto como herramienta de diagnóstico, y el monitoreo de agentes y flujos complejos.

Bloque de optimización y detección (secciones 06 y 07): cómo usar las métricas para optimizar las arquitecturas de contexto en producción, y cómo detectar y responder a la degradación y deriva del sistema.

Bloque de operación (secciones 08 y 09): los dashboards de operación en producción y los patrones y anti-patrones de observabilidad que la experiencia ha validado.

Bloque de síntesis (secciones 10 a 15): caso de estudio empresarial, laboratorio práctico, checklist, resumen, autoevaluación y transición al capítulo de seguridad y gobernanza.

La diferencia entre monitoreo y observabilidad

Antes de continuar, vale la pena establecer una distinción terminológica que tiene consecuencias prácticas. Monitoreo y observabilidad no son sinónimos.

El monitoreo consiste en recolectar métricas predefinidas sobre el comportamiento del sistema. Se sabe de antemano qué se quiere medir y se instrumenta el sistema para producir esas métricas. Si el sistema falla de una manera no prevista, el monitoreo puede no detectarla porque no se definió la métrica correcta.

La observabilidad es una propiedad del sistema: la capacidad de inferir el estado interno del sistema a partir de sus salidas externas. Un sistema observable permite al ingeniero hacer preguntas que no se formularon en el momento del diseño. Cuando aparece un problema nuevo, el ingeniero puede investigar qué ocurrió sin necesidad de rediseñar la instrumentación.

En sistemas de IA, donde los modos de falla son difíciles de anticipar —el modelo puede fallar de formas que nadie predijo cuando lo diseñó—, la observabilidad es más valiosa que el monitoreo puntual. El objetivo no es solo medir lo que ya se sabe que puede fallar; es tener la capacidad de investigar lo que falla de formas inesperadas.

Nota del arquitecto

Un error frecuente en los primeros despliegues de sistemas de IA es tratar la observabilidad como una tarea posterior al lanzamiento: "lo instrumentamos cuando el sistema esté en producción". Esta secuencia produce sistemas que son fundamentalmente opacos en el momento en que más se necesita visibilidad —las primeras semanas de operación real—. La instrumentación que no se diseña antes del lanzamiento tarda semanas en implementarse correctamente, semanas durante las cuales el sistema puede estar fallando de maneras que nadie detecta.

La observabilidad debe diseñarse junto con el sistema, no después. Cada decisión de arquitectura tiene consecuencias para la capacidad de observar: si el sistema no registra qué documentos recuperó en cada llamada, la segunda dimensión de observabilidad es imposible de implementar retroactivamente sin modificar el sistema.

La siguiente sección establece el catálogo de métricas concreto: qué se mide en cada una de las cuatro dimensiones, cómo se calcula y qué umbrales son típicos en sistemas de producción.

Capítulo 13 — Observabilidad, Evaluación y Optimización

Sección 02: Métricas de calidad y desempeño

Una métrica es útil solo si está vinculada a una decisión. Una colección de números sin un criterio de interpretación no es observabilidad; es ruido con formato de dashboard. Antes de instrumentar un sistema de IA, el AI Engineer debe responder dos preguntas para cada métrica que planea recolectar: ¿qué indica esta métrica cuando es alta? ¿qué acción se toma cuando cruza un umbral?

Esta sección cataloga las métricas de los sistemas de IA en dos categorías: métricas operacionales, que miden el comportamiento técnico del sistema, y métricas de calidad, que miden el valor que el sistema entrega. Ambas categorías son necesarias. Ninguna es suficiente por sí sola.

Métricas operacionales

Las métricas operacionales describen cómo se comporta el sistema como infraestructura técnica. Son análogas a las métricas de cualquier sistema distribuido y, en su mayoría, pueden recolectarse con herramientas de observabilidad estándar.

Tiempo al primer token (TTFT, Time To First Token). El tiempo que transcurre desde que la solicitud llega al sistema hasta que el modelo comienza a generar su respuesta. Es la métrica de latencia percibida más relevante para el usuario en sistemas con streaming. Un TTFT alto —típicamente por encima de 2-3 segundos en aplicaciones interactivas— produce la percepción de sistema lento aunque el total de tokens se genere rápidamente. El TTFT depende de la latencia de red, el tiempo de procesamiento previo a la inferencia (recuperación RAG, ensamblado del contexto) y la carga del proveedor del modelo.

Tokens por segundo (TPS, Tokens Per Second). La velocidad de generación de tokens una vez iniciada la respuesta. Determina si el streaming de la respuesta se percibe fluido o interrumpido. Los modelos grandes son inherentemente más lentos que los modelos pequeños; la elección del modelo tiene un efecto directo en esta métrica.

Latencia de extremo a extremo (p50, p95, p99). El tiempo total desde la solicitud hasta la respuesta completa, en percentiles. El p50 describe el caso típico; el p95 y p99 describen los casos lentos que, aunque infrecuentes, determinan la experiencia de los usuarios más afectados. En sistemas de IA, la cola de distribución de latencia tiende a ser más larga que en sistemas tradicionales porque las solicitudes largas o los contextos complejos generan respuestas mucho más largas que la mediana.

Tokens por solicitud (input y output). Cuántos tokens consume el contexto de entrada (system prompt, historial, documentos recuperados, consulta del usuario) y cuántos produce la respuesta. Esta métrica tiene dos usos: calcular el costo por solicitud y detectar anomalías. Un pico inusual en los tokens de entrada puede indicar que el sistema RAG está recuperando demasiados documentos, que el historial de conversación creció sin control, o que se inyectó contenido inesperado en el contexto.

Costo por solicitud y costo por usuario activo. El costo en dólares de cada llamada al modelo, calculado a partir del número de tokens y el precio del proveedor. En sistemas de IA de producción, el costo es una restricción operativa real. Un sistema que aumenta su volumen diez veces sin que el costo escale proporcionalmente indica eficiencias en el uso del contexto. Un sistema cuyo costo escala más que el volumen indica un problema de context bloat que debe investigarse.

Tasa de errores técnicos. El porcentaje de solicitudes que fallan por razones técnicas: límites de la API del proveedor (rate limits), errores de red, timeouts, contextos que superan la ventana máxima del modelo. Distinguir el tipo de error es importante: un pico de errores por rate limit indica que el sistema necesita implementar backoff o distribuir la carga; un pico de errores por contexto demasiado largo indica que la estrategia de construcción del contexto está generando prompts que exceden la capacidad del modelo.

Métricas de calidad

Las métricas de calidad miden si las respuestas del sistema son buenas, no solo si el sistema responde. Son más difíciles de recolectar que las métricas operacionales porque la calidad de una respuesta de lenguaje natural no puede calcularse con una fórmula aritmética. Sin embargo, existen aproximaciones prácticas que permiten estimar la calidad a escala.

Relevancia de la respuesta. En qué medida la respuesta del sistema aborda la consulta del usuario. Una respuesta relevante responde lo que se preguntó. Una respuesta irrelevante puede ser factualmente correcta sobre un tema diferente al que se preguntó. La relevancia

puede estimarse automáticamente usando modelos de scoring o mediante evaluación humana.

Fundamentación en el contexto (groundedness). En qué medida la respuesta está soportada por el conocimiento disponible en el contexto que recibió el modelo. Una respuesta groundada extrae información del contexto; una respuesta no groundada introduce información que no estaba en el contexto —lo que en el dominio de IA se denomina alucinación—. Esta métrica es especialmente crítica en sistemas RAG: si el sistema recupera documentos correctos pero el modelo genera respuestas que no se derivan de esos documentos, hay un problema de groundedness.

Exactitud factual. En qué medida las afirmaciones de la respuesta son correctas respecto al conocimiento real. La exactitud factual es la métrica más difícil de medir automáticamente porque requiere una referencia de verdad de la que muchas veces no se dispone. En sistemas con un dominio acotado y bien documentado —manuales de productos, políticas corporativas, bases de conocimiento técnico— la exactitud puede evaluarse comparando la respuesta contra las fuentes autorizadas.

Coherencia y completitud. Si la respuesta es internamente consistente (no se contradice entre sí), si tiene el nivel de detalle apropiado para la consulta (ni demasiado escueta ni innecesariamente extensa), y si respeta las restricciones de formato y tono definidas en el system prompt.

Satisfacción del usuario. La métrica de calidad más directa es si el usuario quedó satisfecho con la respuesta. Esta puede aproximarse de varias formas: la tasa de conversaciones que terminan sin que el usuario reformule o escale, las calificaciones explícitas si el sistema las solicita (thumbs up/thumbs down), el ratio de sesiones que continúan después de la primera respuesta como señal de engagement.

El problema de las métricas universales de calidad

Una advertencia importante: las métricas de calidad no son universales. La calidad correcta para un asistente de atención al cliente no es la misma que para un asistente de análisis legal ni para un generador de código. El criterio de "relevancia" en un sistema de soporte técnico exige respuestas directas y accionables; en un sistema de análisis de contratos, exige exhaustividad y matices.

El AI Engineer no puede tomar prestado un conjunto de métricas de otro proyecto y aplicarlo sin adaptación. El proceso correcto es:

1. Definir con el equipo de negocio qué significa "una buena respuesta" para este caso de uso específico.
2. Traducir esa definición en criterios observables.
3. Diseñar las métricas que operacionalizan esos criterios.
4. Establecer los umbrales de alerta en función de los datos reales del sistema en producción, no de valores abstractos.

Tabla de métricas de referencia

Métrica	Categoría	Cómo se mide	Umbral de alerta típico
TTFT	Operacional	Timestamp primera respuesta - timestamp solicitud	> 3 segundos en aplicaciones interactivas
Latencia p95	Operacional	Percentil 95 del tiempo de respuesta completo	Varía por caso de uso; establecer en baseline
Tokens de entrada p95	Operacional	Tokens del prompt en percentil 95	> 80% de la ventana de contexto del modelo
Tasa de errores técnicos	Operacional	Errores / solicitudes totales	> 1% en producción
Costo por solicitud	Operacional	Tokens × precio del proveedor	Desviación > 20% del baseline
Relevancia	Calidad	Evaluación automática o humana (0-1)	< 0.80 en promedio semanal
Groundedness	Calidad	Evaluación automática (0-1)	< 0.85 en promedio semanal
Satisfacción del usuario	Calidad	Rating explícito o señal implícita	Caída > 10 puntos desde el baseline

Los valores de la columna "Umbral de alerta típico" son puntos de partida, no verdades absolutas. El equipo debe calibrarlos contra el comportamiento observado del sistema en las primeras semanas de producción.

El scorecard operativo de un sistema de IA

La forma más práctica de gestionar estas métricas en un equipo de operaciones es un scorecard: una vista condensada que muestra el estado del sistema en cada dimensión, con un

indicador de tendencia (mejorando, estable, deteriorando) y un flag de alerta cuando alguna métrica cruza su umbral.

SCORECARD SEMANAL — Sistema RAG de Soporte al Cliente

DIMENSIÓN OPERACIONAL

Latencia p50:	820ms	↔ (estable)
Latencia p95:	2,150ms	↑ (subiendo — revisar)
Tasa de errores:	0.3%	↔ (estable)
Costo/solicitud:	\$0.004	↔ (estable)

DIMENSIÓN DE CALIDAD

Relevancia:	0.83	↔ (estable)
Groundedness:	0.78	↓ (ALERTA — cayó 0.08 pts)
Satisfacción:	72%	↓ (cayó 5 pts — investigar)

INCIDENTES ESTA SEMANA

Martes 14:30 — Pico de errores rate limit (6 minutos)

Jueves 09:00 — 3 respuestas con groundedness < 0.50 detectadas

Este formato permite que el equipo de operaciones identifique en segundos qué está funcionando y qué requiere atención. Las secciones siguientes describen cómo se producen los valores de calidad de este scorecard: a través de pipelines de evaluación automática y humana.

Nota del arquitecto

El error más común en la definición de métricas no es elegir las métricas incorrectas —eso se puede corregir con el tiempo—. Es no conectar las métricas con acciones. Si el equipo define "groundedness < 0.80" como umbral de alerta pero no tiene un proceso definido de respuesta para cuando ese umbral se cruce, la alerta es ruido. Cada umbral de alerta debe tener asociada una acción: quién investiga, con qué prioridad, qué playbook sigue.

El diseño de las métricas y el diseño de los procesos de respuesta son inseparables. Un scorecard sin procesos de respuesta es un tablero decorativo.

La siguiente sección describe cómo producir los valores de las métricas de calidad: las técnicas de evaluación automática y humana que permiten medir, a escala, si las respuestas del sistema son buenas.

Capítulo 13 — Observabilidad, Evaluación y Optimización

Sección 03: Evaluación automática y evaluación humana

Medir la calidad de las respuestas de un sistema de IA a escala es uno de los problemas más abiertos de la ingeniería de IA aplicada. Un sistema de producción puede recibir diez mil solicitudes por día. Revisar cada respuesta manualmente no es factible. Pero delegar toda la evaluación a un algoritmo automático introduce sesgos y limitaciones que pueden ocultar problemas reales.

La solución práctica no es elegir entre evaluación automática y evaluación humana; es diseñar un pipeline que usa cada tipo de evaluación donde tiene más valor y las combina de manera inteligente.

Evaluación automática: LLM-as-judge

La técnica más adoptada para la evaluación automática de respuestas de sistemas de IA es el LLM-as-judge: usar un modelo de lenguaje como evaluador de las respuestas de otro modelo. La idea es directa. Si se tiene una consulta, el contexto que recibió el sistema y la respuesta que generó, un segundo modelo puede evaluar si esa respuesta es relevante, está fundamentada en el contexto y es factualmente correcta.

La implementación básica tiene tres componentes:

El prompt de evaluación. Define el criterio que el modelo evaluador debe aplicar. Por ejemplo:

```
Eres un evaluador de calidad para un sistema de soporte técnico.  
Dada la consulta del usuario, el contexto disponible y la respuesta generada,  
evalúa la respuesta en tres dimensiones:  
  
1. RELEVANCIA (0-1): ¿La respuesta aborda directamente la consulta del usuario?  
2. FUNDAMENTACIÓN (0-1): ¿Cada afirmación de la respuesta puede verificarse en el conte  
3. COMPLETITUD (0-1): ¿La respuesta incluye toda la información del contexto que es rel  
  
Proporciona un score numérico para cada dimensión y una justificación breve.  
  
CONSULTA: {consulta}  
CONTEXTO: {contexto_recuperado}  
RESPUESTA: {respuesta_del_sistema}
```

El modelo evaluador. Puede ser el mismo modelo que produce las respuestas o un modelo diferente. En general, usar un modelo más capaz o diferente como evaluador reduce el riesgo de que el evaluador favorezca sistemáticamente el estilo de respuesta del modelo evaluado.

El pipeline de ejecución. El proceso automatizado que toma una muestra de las solicitudes de producción, construye el prompt de evaluación para cada una y ejecuta el modelo evaluador. Los scores resultantes alimentan las métricas de calidad del sistema.

Limitaciones conocidas del LLM-as-judge

El LLM-as-judge es una herramienta útil pero tiene limitaciones bien documentadas que el AI Engineer debe conocer antes de confiar en los scores que produce.

Sesgo de autpreferencia. Los modelos tienden a puntuar más alto las respuestas que se parecen al estilo que ellos mismos generarían. Si el evaluador y el sistema evaluado son del mismo modelo o de la misma familia de modelos, el evaluador puede calificar bien respuestas que otro evaluador —humano o de otra familia— calificaría peor.

Sesgo de posición. En evaluaciones donde se comparan dos respuestas (A/B), los modelos tienden a favorecer la respuesta que aparece primero, independientemente de su calidad real. Para mitigar este sesgo, las comparaciones deben ejecutarse en ambos órdenes y promediar los resultados.

Sensibilidad al formato del prompt de evaluación. Cambios menores en la redacción del prompt de evaluación pueden producir scores significativamente diferentes. Un prompt que pide "evalúa si la respuesta es correcta" produce resultados distintos a uno que pide "evalúa si cada afirmación de la respuesta está soportada por el contexto". El equipo debe validar que el prompt de evaluación captura el criterio de calidad correcto para el caso de uso.

Incapacidad para detectar alucinaciones sutiles. Un modelo evaluador que recibe el mismo contexto que el sistema evaluado puede confirmar que la respuesta es consistente con ese contexto —lo que es la definición de groundedness— sin detectar si el contexto mismo contiene información desactualizada. La evaluación de la exactitud factual respecto al mundo real, no respecto al contexto local, requiere una fuente de verdad externa.

Costo adicional. Ejecutar un modelo evaluador por cada respuesta de producción duplica o triplica el costo por solicitud si se evalúa el 100% del tráfico. En la práctica, el LLM-as-judge se ejecuta sobre una muestra del tráfico —típicamente el 5-20%— con muestreo estratificado para asegurar representatividad.

Evaluación humana: cuándo es indispensable

La evaluación humana no puede reemplazarse en tres situaciones.

Primera: calibración del evaluador automático. Antes de confiar en los scores del LLM-as-judge, el equipo debe calibrarlo contra evaluaciones humanas. Si evaluadores humanos expertos puntúan una muestra de respuestas con consistencia, y los scores del modelo evaluador correlacionan fuertemente con esos scores humanos, el evaluador automático es confiable para esa distribución de casos. Si la correlación es baja, el evaluador automático tiene un sesgo sistemático que debe corregirse.

Segunda: casos de alta consecuencia. En dominios donde una respuesta incorrecta tiene consecuencias graves —diagnósticos médicos, asesoría legal, decisiones financieras—, la evaluación humana de una fracción representativa de las respuestas es una salvaguarda no negociable. Los modelos evaluadores no asumen responsabilidad por los errores que no detectan.

Tercera: nuevos patrones de falla. Cuando el sistema se enfrenta a un tipo de consulta que no estaba presente en los datos de calibración del evaluador automático, los scores del modelo evaluador pueden ser no confiables para ese nuevo patrón. El equipo de operaciones debe revisar manualmente los casos en que el evaluador automático produce scores extremos

—muy altos o muy bajos— para detectar si hay un nuevo modo de falla que el evaluador no está capturando correctamente.

El proceso de evaluación humana estructurado

Cuando se conduce evaluación humana, la estructura del proceso determina la calidad de los resultados. La evaluación informal —"alguien en el equipo revisa algunas respuestas"— produce datos que son difíciles de agregar y comparar en el tiempo.

Un proceso estructurado de evaluación humana tiene cuatro elementos:

Protocolo de evaluación. Un documento que define qué criterios se evalúan, cómo se escalan (qué significa un 3/5 en relevancia versus un 4/5), qué ejemplos de cada nivel sirven de referencia para los evaluadores.

Selección de la muestra. Las respuestas a evaluar deben seleccionarse de forma que sean representativas del tráfico real. Una muestra sesgada hacia los casos fáciles produce una estimación de calidad optimista; una muestra sesgada hacia los casos difíciles produce el efecto contrario.

Múltiples evaluadores por caso. Para medir la consistencia humana —y por tanto la confiabilidad de las evaluaciones—, al menos un subconjunto de los casos debe ser evaluado por dos o más personas. Si hay desacuerdo frecuente entre evaluadores, el protocolo de evaluación no es suficientemente preciso o el criterio de calidad es intrínsecamente subjetivo.

Medición de acuerdo entre evaluadores (inter-rater reliability). Una métrica como el coeficiente kappa de Cohen cuantifica en qué medida los evaluadores humanos están de acuerdo entre sí. Un kappa mayor de 0.7 indica acuerdo sustancial; por debajo de 0.5, el proceso de evaluación produce datos poco confiables.

Datasets de evaluación: el golden set

Una práctica fundamental en sistemas de IA maduros es el mantenimiento de un golden set: un conjunto de casos de referencia con respuestas de calidad conocida, construido y validado por expertos del dominio.

El golden set sirve para dos propósitos distintos.

Primero, permite evaluar el sistema de forma repetible a lo largo del tiempo. Si el sistema pasa los casos del golden set con calidad alta hoy y con calidad más baja en tres meses, hay un deterioro detectable que puede investigarse.

Segundo, permite comparar versiones del sistema. Cuando se cambia el system prompt, se actualiza la base vectorial o se cambia el modelo, ejecutar el golden set sobre la nueva versión y comparar los resultados con la versión anterior produce evidencia objetiva de si el cambio mejoró o deterioró la calidad del sistema.

Un golden set efectivo incluye:

- Casos representativos del uso normal del sistema
- Casos borde conocidos: consultas que en el pasado produjeron respuestas incorrectas o insatisfactorias
- Casos adversariales: consultas diseñadas para detectar comportamientos problemáticos (instrucciones que contradicen el system prompt, preguntas fuera del dominio, consultas con información incorrecta en la premisa)
- Casos que verifican restricciones críticas: el sistema nunca debe revelar información confidencial, nunca debe contradecir una política corporativa específica, nunca debe responder fuera de su dominio declarado

El pipeline de evaluación combinado

El pipeline de evaluación que funciona en producción combina las evaluaciones automáticas y humanas en un sistema cohesivo:

PIPELINE DE EVALUACIÓN — Operación continua

[PRODUCCIÓN] → 100% del tráfico

|

└→ [MÉTRICAS OPERACIONALES] → Latencia, tokens, errores

|

(recolección automática, tiempo real)

|

└→ [LLM-AS-JUDGE] → Relevancia, groundedness, coherencia

|

(muestra 10-20%, ejecución asíncrona)

|

└→ [GOLDEN SET] → Evaluación periódica (semanal/quincenal)

(todos los casos, con comparación de versiones)

[EVALUACIÓN HUMANA] → Flujo paralelo

|

└→ Calibración del LLM-as-judge (mensual)

└→ Revisión de casos extremos (continua)

└→ Análisis de casos de alta consecuencia (por incidente)

[AGREGACIÓN] → Scorecard semanal

Métricas operacionales + scores de calidad + tendencias

Nota del arquitecto

La evaluación de sistemas de IA es un campo en evolución activa. Las técnicas que se describen aquí representan el estado práctico consolidado, pero no son definitivas. Investigaciones recientes muestran que los evaluadores automáticos basados en modelos de lenguaje tienen sesgos sistemáticos que pueden conducir a conclusiones erróneas sobre la calidad del sistema. La calibración periódica contra evaluadores humanos no es opcional; es la única forma de saber si los números que el pipeline automático produce son confiables.

Un equipo que confía ciegamente en los scores del LLM-as-judge sin calibración humana regular corre el riesgo de optimizar el sistema para satisfacer al evaluador —un fenómeno análogo a Goodhart's Law: cuando una métrica se convierte en el objetivo, deja de ser una buena métrica—.

La siguiente sección introduce la trazabilidad de prompts y contexto: cómo registrar no solo la respuesta del sistema sino el contexto completo que el modelo recibió, para poder diagnosticar respuestas incorrectas con precisión quirúrgica.

Capítulo 13 — Observabilidad, Evaluación y Optimización

Sección 04: Trazabilidad de prompts y contexto

Cuando un sistema de IA produce una respuesta incorrecta, la pregunta de diagnóstico inmediata es: ¿qué tenía el modelo disponible en ese momento? La respuesta a esa pregunta no es la consulta del usuario. Es el contexto completo que se ensambló y se envió al modelo: el system prompt en su versión específica de producción, los documentos recuperados por el sistema RAG con sus metadatos, el historial de conversación que se incluyó, los resultados de las herramientas que el agente ejecutó, y la consulta reformateada del usuario.

La trazabilidad de contexto es la capacidad de reconstruir ese contexto completo para cualquier solicitud de producción, en cualquier momento posterior. Es la segunda dimensión de observabilidad definida en la sección 01, y es la más diferenciadora: sin ella, el diagnóstico de respuestas incorrectas depende de conjeturas; con ella, el diagnóstico es reproducible y preciso.

Por qué la trazabilidad de contexto es distinta al logging tradicional

En el software tradicional, el logging registra eventos: una función fue llamada con estos parámetros, una base de datos ejecutó esta consulta, un servicio respondió con este código HTTP. Los logs son registros de lo que hizo el sistema.

En sistemas de IA, lo que hizo el sistema —enviaron una consulta al modelo y recibieron una respuesta— no es suficiente para el diagnóstico. Lo que importa es con qué información exacta lo hizo. El mismo código de sistema puede producir respuestas completamente diferentes si la base vectorial tiene documentos actualizados o desactualizados, si el system prompt fue modificado la semana pasada, si la consulta del usuario activó un patrón de recuperación diferente.

La trazabilidad de contexto registra no la acción sino el estado del mundo en el momento de la acción: el contexto completo que el modelo recibió.

Los componentes de una traza de contexto

Una traza completa de contexto tiene siete componentes que deben registrarse para que el diagnóstico sea posible.

Identificador de traza. Un ID único que vincula todos los registros de una solicitud específica a través del sistema. En pipelines multi-etapa —consulta del usuario → reformulación → recuperación RAG → generación → validación— el identificador de traza permite reconstruir el flujo completo aunque cada etapa esté en un servicio diferente.

Versión del system prompt. No el contenido del system prompt, sino el identificador de su versión en el sistema de control de versiones. Con la versión registrada, el equipo puede recuperar exactamente cuáles instrucciones estaban activas en el momento de la solicitud, incluso semanas después. Esto es crítico para diagnósticos retroactivos: si el sistema comenzó a fallar el martes pasado, la pregunta "¿cambió el system prompt ese día?" tiene respuesta objetiva.

Consulta del usuario. La consulta original, posiblemente reformulada o expandida por el sistema antes de usarla en la recuperación. Registrar ambas —la consulta original y la consulta procesada— permite detectar casos donde la reformulación introduce sesgos en la recuperación.

Documentos recuperados. Para sistemas RAG, la lista de documentos que el sistema recuperó, incluyendo: identificador del documento, nombre o URL de la fuente, fragmento exacto recuperado, score de relevancia asignado por el sistema de recuperación, y metadatos clave (fecha de creación, fecha de última modificación). Esta información permite responder: ¿era correcto el documento que el modelo usó? ¿el score de relevancia fue apropiado? ¿el fragmento recuperado contenía la información necesaria?

Historial de conversación incluido. Cuántos turnos de conversación anterior se incluyeron en el contexto, y cuáles. En conversaciones largas donde el historial se trunca por limitaciones de la ventana de contexto, registrar qué partes se incluyeron y cuáles se omitieron permite diagnosticar casos donde la respuesta incorrecta se debe a que el modelo no tenía acceso a información relevante de turnos anteriores.

Herramientas ejecutadas. Para sistemas agentivos, la secuencia de herramientas que el agente ejecutó: nombre de la herramienta, parámetros de entrada, resultado de la ejecución, y tiempo de ejecución. Si el agente ejecutó una búsqueda en internet, una consulta a una base de datos o una llamada a una API externa, el resultado de esas operaciones forma parte del contexto que el modelo procesó.

Respuesta generada y metadatos de generación. El texto completo de la respuesta del modelo, el número de tokens generados, el tiempo de generación, y el modelo específico que se usó (incluyendo la versión del modelo cuando el proveedor la expone).

Implementación: distributed tracing para sistemas de IA

El concepto técnico más útil para implementar trazabilidad de contexto es el distributed tracing, que proviene de la ingeniería de sistemas distribuidos. En OpenTelemetry —el estándar abierto de observabilidad— una traza es un árbol de spans: cada etapa del procesamiento es un span con su duración, sus atributos y su relación con los spans padre e hijo.

Para un sistema de IA, el árbol de spans tiene una estructura que refleja el pipeline de procesamiento:

```

TRAZA — Solicitud de soporte al cliente (ID: trace-8f2a91)

SPAN RAÍZ: handle_user_request (total: 1,847ms)
|
|— SPAN: preprocess_query (12ms)
|   atributos: {query_original, query_expanded}
|
|— SPAN: retrieve_context (340ms)
|   |
|   |— SPAN: embed_query (45ms)
|   |   atributos: {tokens, modelo_embedding}
|   |
|   |— SPAN: vector_search (295ms)
|   |   atributos: {k=5, scores=[0.91, 0.87, 0.82, 0.78, 0.71],
|   |               doc_ids=[...], fragmentos=[...]}
|   |
|   |— SPAN: assemble_context (8ms)
|   |   atributos: {tokens_sistema: 1200, tokens_recuperados: 2100,
|   |               tokens_historial: 450, tokens_usuario: 85,
|   |               tokens_total: 3835, system_prompt_version: "v2.3.1"}
|   |
|   |— SPAN: call_model (1,420ms)
|   |   atributos: {modelo: "gpt-4o-mini", tokens_input: 3835,
|   |               tokens_output: 312, ttft_ms: 890}
|   |
|   |— SPAN: postprocess_response (67ms)
|   |   atributos: {groundedness_score: 0.89, response_id: "resp-7c4d"}

```

Este árbol de spans permite responder en segundos preguntas que sin trazabilidad requerirían horas de análisis: ¿cuánto tardó la recuperación? ¿qué documentos se usaron? ¿cuántos tokens consumió el historial? ¿qué versión del system prompt estaba activa?

Qué registrar y qué no registrar

La trazabilidad completa tiene un costo: almacenamiento y procesamiento de datos. En un sistema que maneja millones de solicitudes, registrar el contexto completo de cada una puede

ser prohibitivo en costo de almacenamiento. Hay estrategias para gestionarlo.

Muestreo de trazas. Registrar la traza completa de una fracción del tráfico —típicamente el 10-20% para sistemas de volumen moderado, menor para sistemas de muy alto volumen— con muestreo determinístico que asegura que los casos con errores técnicos o scores de calidad bajos siempre se registran. El muestreo inteligente captura los casos normales con menor densidad y los casos anómalos con mayor densidad.

Retención diferenciada. Las trazas de solicitudes normales pueden retenerse por 30 días; las trazas de solicitudes con errores, scores de calidad bajos o respuestas que el usuario marcó explícitamente como incorrectas deben retenerse por períodos más largos (90 días o más) para permitir análisis retrospectivos.

Compresión del contenido. Almacenar hashes de los documentos recuperados en lugar del contenido completo, con la capacidad de recuperar el contenido desde el sistema fuente cuando sea necesario para diagnóstico. Esto reduce el volumen de almacenamiento sin perder la capacidad de reconstruir el contexto.

Consideraciones de privacidad. Los datos del usuario —consultas, fragmentos de conversación— son datos personales en la mayoría de las jurisdicciones. La trazabilidad debe diseñarse desde el inicio con las restricciones de privacidad aplicables: quién puede acceder a las trazas, durante cuánto tiempo se retienen, cómo se anonimiza la información del usuario para análisis agregados.

El valor diagnóstico de la trazabilidad: un ejemplo

Supóngase que el equipo de soporte reporta que el asistente de IA dio una respuesta incorrecta sobre el proceso de devolución de un producto. Sin trazabilidad, el ingeniero debe adivinar qué ocurrió: ¿el sistema prompt era incorrecto? ¿el RAG recuperó el documento equivocado? ¿el modelo alucinó?

Con la traza de esa solicitud específica, el diagnóstico es directo:

- Se recupera el identificador de la solicitud del log de conversación.
- Se busca la traza correspondiente en el sistema de observabilidad.
- Se verifica la versión del system prompt: v2.3.1, actualizada hace tres semanas. No coincide con el problema reportado.
- Se revisan los documentos recuperados: el sistema recuperó un fragmento de la política de devoluciones con fecha de modificación de hace seis meses, que menciona un plazo de 30 días. La política actual, actualizada hace dos semanas, establece un plazo de 15 días.

- El documento desactualizado no fue reindexado después de la actualización de la política.

El diagnóstico tomó tres minutos. La solución —reindexar el documento actualizado y verificar si hay otros documentos con el mismo problema— tomó una hora. Sin trazabilidad, el mismo diagnóstico habría requerido revisar manualmente múltiples fuentes posibles de error, y podría haber llegado a la conclusión incorrecta de que el problema era el modelo prompt.

Nota del arquitecto

La trazabilidad de contexto es el componente de observabilidad más diferenciador y, paradójicamente, el más frecuentemente omitido en los primeros diseños de sistemas de IA. La razón es que su valor no es visible cuando el sistema funciona bien; solo se vuelve evidente cuando hay un problema que diagnosticar. La tentación es omitirla para reducir la complejidad inicial y agregarla después "si es necesario".

El problema es que agregar trazabilidad retroactivamente requiere modificar el sistema, lo que puede ser costoso y disruptivo. Un sistema de IA sin trazabilidad diseñada desde el inicio es un sistema que opera en modo opaco: puede funcionar bien, puede funcionar mal, y en ambos casos el equipo tiene poca capacidad de saberlo con certeza.

Diseñar la trazabilidad desde el primer sprint es inversión, no gasto. El retorno aparece la primera vez que hay que diagnosticar un problema de producción.

La siguiente sección extiende la observabilidad desde las llamadas individuales al modelo hacia los flujos completos de agentes multi-etapa: cómo monitorear ciclos agentivos, detectar bucles y medir la latencia por etapa de un pipeline complejo.

Capítulo 13 — Observabilidad, Evaluación y Optimización

Sección 05: Monitoreo de agentes y flujos

El monitoreo de una llamada individual al modelo es relativamente directo: hay una entrada, una salida, una latencia medible y un costo calculable. El monitoreo de un sistema agentivo es cualitativamente más complejo porque el agente no produce una respuesta en un solo paso. Produce un flujo de acciones —planificación, ejecución de herramientas, observación de resultados, replaneación— que puede tener decenas de pasos, durar minutos, y fallar de maneras que ningún paso individual indica de forma aislada.

Esta sección describe cómo extender la observabilidad a los sistemas agentivos: qué métricas adicionales se necesitan, cómo detectar los modos de falla específicos de los agentes y cómo visualizar flujos de múltiples etapas de manera que el equipo de operaciones pueda entender qué está ocurriendo.

Los modos de falla específicos de los agentes

Un agente bien implementado tiene mecanismos para hacer las preguntas correctas, usar las herramientas disponibles y llegar a una respuesta útil. Pero tiene también modos de falla que no aparecen en sistemas de inferencia directa.

El bucle infinito. El agente entra en un ciclo donde repite las mismas acciones sin progresar hacia una respuesta. Puede ocurrir cuando el agente interpreta los resultados de una herramienta como evidencia de que debe ejecutar la misma herramienta nuevamente, o cuando el criterio de terminación del ciclo agentivo no está bien definido. Sin monitoreo, un bucle infinito consume tokens y costo de forma indefinida hasta que el sistema alcanza un timeout externo o agota el presupuesto.

La explosión de herramientas. El agente llama a muchas más herramientas de las necesarias para responder la consulta. En lugar de responder una pregunta con dos o tres herramientas relevantes, llama a diez o quince herramientas en búsqueda de certeza adicional. Esto aumenta la latencia, el costo y a veces produce respuestas peores porque el

contexto acumulado de múltiples herramientas es más difícil de sintetizar que el de pocas herramientas precisas.

El abandono prematuro. El agente decide que no puede responder la consulta y se detiene antes de haber explorado las herramientas que habrían permitido una respuesta correcta. Puede ocurrir cuando el primer intento de recuperación no produce resultados relevantes y el agente interpreta eso como una señal de que el sistema no tiene la información, cuando en realidad una estrategia de búsqueda diferente habría encontrado la información.

La propagación de errores. Una herramienta del pipeline produce un resultado incorrecto o ambiguo, y el agente lo incorpora en su razonamiento sin cuestionar su validez. Los pasos posteriores se construyen sobre una base incorrecta, produciendo una respuesta final que es incorrecta de forma no obvia —el razonamiento parece coherente pero parte de una premisa falsa—.

La divergencia de objetivos. El agente pierde de vista el objetivo original de la consulta a medida que avanza en el proceso de razonamiento. Las herramientas que ejecuta en los pasos finales responden una pregunta diferente a la que se hizo al inicio. Esto ocurre con mayor frecuencia en agentes con muchos pasos de razonamiento donde el contexto de la consulta original se diluye.

Métricas específicas para sistemas agentivos

Además de las métricas operacionales estándar, los sistemas agentivos requieren métricas adicionales que capturan la dinámica de los flujos multi-etapa.

Número de pasos por solicitud. Cuántas iteraciones del ciclo agentivo (planificación → acción → observación) se ejecutaron para resolver la consulta. Una distribución estable con un máximo razonable indica un agente bien controlado. Una distribución con cola larga o con solicitudes que alcanzan el máximo configurado indica que el agente está entrando en bucles o tomando caminos demasiado indirectos.

Número de herramientas llamadas por solicitud. Cuántas herramientas distintas ejecutó el agente, y cuántas veces llamó a la misma herramienta. Si el agente llama a la misma herramienta con parámetros similares tres veces seguidas, es señal de un bucle o de un criterio de terminación mal definido.

Tasa de éxito por herramienta. El porcentaje de llamadas a cada herramienta que producen un resultado útil (no un error, no un resultado vacío). Una herramienta con alta tasa de fallos es un punto de fragilidad del sistema que debe ser reforzado o reemplazado.

Latencia por etapa del pipeline. Cuánto tiempo consume cada etapa: planificación inicial, cada llamada a herramienta, cada paso de razonamiento, síntesis de la respuesta final. Esta descomposición permite identificar dónde están los cuellos de botella: si el 80% de la latencia está en una herramienta específica, ese es el lugar donde optimizar.

Costo acumulado por flujo. El costo total de un flujo agentivo completo, que puede ser significativamente mayor que el costo de una llamada individual. En sistemas con costos por token, un agente que da diez pasos tiene un costo de inferencia diez veces mayor que uno que da un paso. Si el costo promedio por solicitud de un agente está creciendo, significa que los flujos se están volviendo más complejos con el tiempo —lo que puede ser esperado o puede ser una degradación que requiere investigación—.

Tasa de terminación exitosa. El porcentaje de flujos agentivos que terminan con una respuesta (éxito) versus los que terminan por timeout, por error técnico, o porque el agente declaró que no podía responder. Una tasa de terminación exitosa que cae indica que el agente está encontrando más situaciones en las que no puede operar, lo que puede señalar cambios en el tipo de consultas que recibe o deterioro en las herramientas disponibles.

Visualización de flujos agentivos

El formato de log plano no es adecuado para sistemas agentivos. Una secuencia lineal de eventos no comunica la estructura de árbol del razonamiento. El formato correcto es el árbol de spans descrito en la sección anterior, con la adición de visualizaciones que muestren el flujo como un grafo de decisiones.

FLUJO AGENTIVO — Solicitud de análisis de inventario (ID: agent-3d7f)

[PLANIFICACIÓN INICIAL]

Objetivo: "¿Qué productos del catálogo tienen stock bajo para la campaña de verano?"

Plan inicial: 1) consultar inventario → 2) filtrar productos de temporada → 3) compar

[PASO 1] → tool: query_inventory_db

Parámetros: {categoria: "verano", umbral_stock: null}

Resultado: 847 productos devueltos

Latencia: 420ms

Decisión: demasiados resultados, refinar búsqueda

[PASO 2] → tool: query_inventory_db (segunda llamada)

Parámetros: {categoria: "verano", umbral_stock: 50}

Resultado: 23 productos con stock <= 50 unidades

Latencia: 380ms

Decisión: obtener predicción de demanda para estos 23 productos

[PASO 3] → tool: get_demand_forecast

Parámetros: {product_ids: [23 IDs], periodo: "verano-2026"}

Resultado: predicción por producto con intervalos de confianza

Latencia: 1,240ms

Decisión: suficiente información, sintetizar respuesta

[SÍNTESIS FINAL]

Duración total: 2,980ms

Pasos ejecutados: 3

Herramientas usadas: 2 (query_inventory_db × 2, get_demand_forecast × 1)

Tokens totales: 8,450

Resultado: respuesta con 8 productos en riesgo de desabastecimiento

Esta representación permite que el equipo de operaciones vea de un vistazo si el flujo fue directo o estuvo exploración innecesaria, cuánto tiempo tardó cada herramienta y cuál fue la secuencia de decisiones del agente.

Detección de bucles y comportamientos anómalos

Para detectar bucles y comportamientos anómalos en tiempo real, el sistema de monitoreo debe implementar alertas específicas:

Alerta de pasos excesivos. Si un flujo agentivo supera un umbral configurado de pasos — por ejemplo, 15 pasos en un agente diseñado para operar en 3-7— la alerta se dispara automáticamente. La solicitud puede continuar mientras se investiga, o puede interrumpirse si el costo proyectado es inaceptable.

Alerta de herramienta repetida. Si el agente llama a la misma herramienta con los mismos parámetros más de una vez en el mismo flujo, es señal de un bucle potencial. El sistema puede alertar al equipo y opcionalmente interrumpir el flujo.

Alerta de costo proyectado. Si el costo acumulado de un flujo supera el percentil 95 del costo histórico de solicitudes similares, la alerta indica que este flujo es anómalamente costoso y merece inspección.

Detección de ciclos en el grafo de decisiones. Un análisis automático del árbol de spans puede detectar si el agente visitó el mismo estado (herramienta + parámetros similares) más de una vez, lo que es evidencia de un ciclo en la lógica de planificación.

Monitoreo de sistemas multi-agente

Los sistemas de múltiples agentes coordinados —donde un agente orquestador delega tareas a agentes especializados— presentan un nivel adicional de complejidad. Cada agente tiene su propio ciclo de vida y su propia traza, pero las trazas deben vincularse para poder entender el flujo completo de una solicitud.

El mecanismo correcto es propagar el identificador de traza desde el orquestador a cada subagente. Cuando el subagente genera su traza, la registra como hija de la traza del orquestador. El resultado es un árbol de trazas anidadas que muestra el flujo completo del sistema, desde la solicitud del usuario hasta la respuesta final, pasando por todas las delegaciones y subagentes involucrados.

Sin esta propagación de identificadores, las trazas de los subagentes quedan huérfanas: existen como registros independientes, pero no pueden vincularse con la solicitud que los originó.

Nota del arquitecto

El monitoreo de sistemas agentivos requiere herramientas diseñadas para ese propósito. Las herramientas de monitoreo de software tradicional pueden capturar las métricas operacionales, pero no pueden visualizar árboles de razonamiento, detectar bucles en flujos agentivos o mostrar la secuencia de herramientas ejecutadas de manera comprensible. Plataformas especializadas como Langfuse, LangSmith, Phoenix y Weights & Biases ofrecen visualizaciones adaptadas a sistemas de IA. La elección de plataforma es secundaria al principio: el sistema debe instrumentarse para producir los datos que esas plataformas necesitan —identificadores de traza, spans jerárquicos, atributos de contexto—. Una vez que el sistema produce esos datos, migrar de plataforma es relativamente sencillo. Un sistema que no los produce no puede beneficiarse de ninguna plataforma.

La siguiente sección examina cómo usar los datos de observabilidad para optimizar las arquitecturas de contexto en producción: el ciclo de experimentación que permite mejorar el sistema basándose en evidencia de uso real.

Capítulo 13 — Observabilidad, Evaluación y Optimización

Sección 06: Optimización continua de prompts y contexto

El Módulo 2 de este libro enseñó a construir prompts efectivos antes del despliegue: cómo estructurar las instrucciones del sistema, cómo usar few-shot examples, cómo diseñar el contexto para maximizar la calidad de la respuesta en condiciones de laboratorio. Esta sección aborda un problema diferente: cómo mejorar un sistema que ya está en producción, usando los datos reales que el sistema genera, para decisiones de optimización que ningún experimento de laboratorio puede replicar.

La diferencia es fundamental. En laboratorio, el ingeniero controla la distribución de consultas. En producción, los usuarios hacen las preguntas que tienen, no las que el ingeniero esperaba. Las consultas reales revelan áreas débiles del sistema que los conjuntos de prueba no cubren, patrones de uso no anticipados que la arquitectura original no optimizó, y oportunidades de mejora que solo son visibles cuando hay suficiente volumen de datos.

El ciclo de optimización continua

La optimización continua de un sistema de IA en producción sigue un ciclo de cuatro etapas que se repiten indefinidamente.

Etapas 1: Observar. Recolectar las métricas de las cuatro dimensiones de observabilidad. Identificar los patrones anómalos: qué tipos de consultas producen scores de calidad más bajos, qué fragmentos de documentos se recuperan con mayor frecuencia y producen respuestas incorrectas, qué categorías de preguntas el agente resuelve con más pasos de los esperados, qué parte del día o de la semana concentra los picos de insatisfacción del usuario.

Etapas 2: Formular hipótesis. Traducir la observación en una hipótesis de causa y en una propuesta de intervención. La observación puede ser "las consultas sobre políticas de devolución tienen groundedness de 0.65, 20 puntos por debajo del promedio". La hipótesis es "el fragmento de documentos de política de devoluciones que se indexó está desactualizado y el sistema recupera información incorrecta". La intervención propuesta es "reindexar el

documento de política de devoluciones actualizado y verificar si hay otros documentos de política con el mismo problema".

Etapla 3: Intervenir y medir. Implementar la intervención en el sistema —puede ser una actualización de la base vectorial, un cambio en el system prompt, un ajuste en el algoritmo de recuperación, o un cambio en el modelo— y medir su impacto usando las métricas de la etapa 1. Para que la medición sea rigurosa, la intervención debe compararse contra una base: el comportamiento del sistema antes del cambio.

Etapla 4: Decidir y documentar. Si la intervención mejoró las métricas objetivo sin degradar otras métricas, se consolida en producción. Si no mejoró, o mejoró una métrica a costa de otra, se evalúa si el trade-off es aceptable o si se necesita una hipótesis diferente. En cualquier caso, se documenta qué se hizo, con qué resultado, para construir una base de conocimiento institucional sobre el comportamiento del sistema.

A/B testing para arquitecturas de contexto

El instrumento más riguroso para comparar dos versiones de una arquitectura de contexto es el A/B testing: exponer una fracción del tráfico de producción a la versión A del sistema (control) y la fracción restante a la versión B (variante), y comparar las métricas entre ambos grupos.

El A/B testing para sistemas de IA tiene particularidades que lo diferencian del A/B testing de interfaces de usuario.

Las métricas de éxito son más ruidosas. Un botón tiene una tasa de clic que se mide directamente. La calidad de una respuesta de IA requiere evaluación —automática o humana— que introduce ruido. Esto significa que los A/B tests de sistemas de IA necesitan muestras más grandes y duraciones más largas para producir resultados estadísticamente significativos.

Los efectos de interferencia son posibles. Si un usuario interactúa con el sistema múltiples veces y es asignado a versiones diferentes en distintas sesiones, su experiencia se contamina. El diseño correcto asigna usuarios —no solicitudes— a los grupos de forma consistente a lo largo del tiempo.

Las métricas secundarias pueden divergir. Una variante que mejora la relevancia puede aumentar la latencia porque el sistema de recuperación más sofisticado es más lento. Las decisiones deben considerar el conjunto de métricas, no solo la métrica primaria del experimento.

Los efectos de aprendizaje del usuario existen. Los usuarios aprenden a usar el sistema de cierta manera. Si la versión B requiere consultas formuladas de forma diferente para obtener mejores resultados, los usuarios de la versión B pueden tardar en adaptar su comportamiento, produciendo una ventaja inicial de la versión A que desaparece con el tiempo.

Qué puede optimizarse con A/B testing

Las dimensiones de la arquitectura de contexto que pueden compararse mediante A/B testing son múltiples:

Estrategias de recuperación. Recuperar los K documentos más relevantes por score versus usar un umbral de score mínimo. Usar solo búsqueda semántica versus combinar búsqueda semántica con búsqueda léxica (BM25). Aplicar re-ranking posterior a la recuperación versus usar el ranking original del buscador vectorial.

Configuración del contexto. Incluir historial de conversación de 3 turnos versus 5 turnos. Incluir un preamble de orientación al modelo antes de los documentos versus no incluirlo. Comprimir el historial de conversación mediante resumen versus truncación directa.

System prompt. Versiones alternativas de las instrucciones del sistema que enfatizan diferentes aspectos de la respuesta esperada. Adición de few-shot examples versus system prompt sin ejemplos. Diferentes niveles de restricción sobre el dominio del asistente.

Modelo. Comparar el mismo sistema con modelos de diferente tamaño o familia. Esto es especialmente relevante para encontrar el punto donde el modelo más pequeño produce calidad suficiente a menor costo.

El problema de la significancia estadística

Un error frecuente en la optimización de sistemas de IA es tomar decisiones basadas en diferencias de métricas que no son estadísticamente significativas. Si la versión A tiene groundedness de 0.83 y la versión B tiene groundedness de 0.85, la diferencia parece favorable a B. Pero si el intervalo de confianza del 95% de ambas métricas se superpone, esa diferencia puede deberse al azar y no a una diferencia real en el sistema.

El principio básico: no consolidar una intervención en producción hasta que la diferencia en las métricas primarias sea estadísticamente significativa. En la práctica, esto significa definir antes del experimento el tamaño de muestra necesario para detectar la diferencia mínima que

se considera relevante para el negocio (el "effect size" mínimo detectable), y no evaluar el experimento hasta que ese tamaño de muestra se alcance.

Para sistemas de IA donde las métricas de calidad son ruidosas, los experimentos necesitan típicamente entre 500 y 2,000 solicitudes evaluadas por variante para producir resultados confiables. Los sistemas de bajo volumen de tráfico pueden requerir semanas para alcanzar ese tamaño de muestra.

Optimización sin experimentación controlada: el análisis de subgrupos

En sistemas con volumen de tráfico insuficiente para A/B testing, o cuando la intervención es urgente y no puede esperar un experimento, el análisis de subgrupos puede guiar la optimización.

El análisis de subgrupos examina las métricas de los distintos tipos de consultas que el sistema recibe. Si las consultas del tipo "¿cuánto cuesta X?" tienen groundedness de 0.91, mientras que las consultas del tipo "¿cuál es la diferencia entre X e Y?" tienen groundedness de 0.69, el análisis de subgrupos indica que el problema de calidad está concentrado en las consultas comparativas, no en el sistema en general.

Este hallazgo puede guiar una intervención específica —por ejemplo, agregar documentos de comparación de productos a la base vectorial o modificar el system prompt para que el sistema solicite más contexto antes de responder consultas comparativas— sin necesitar un A/B test formal.

Nota del arquitecto

La optimización continua no es una actividad que ocurre de vez en cuando cuando el sistema da problemas. Es un proceso regular, con cadencia definida —por ejemplo, una revisión semanal de métricas y una intervención mensual como mínimo— que convierte los datos de observabilidad en mejoras concretas del sistema. Los equipos que no tienen esta cadencia tienden a reaccionar solo cuando los problemas son graves, en lugar de detectarlos y corregirlos cuando son menores.

La optimización continua también requiere una cultura de experimentación honesta: estar dispuesto a concluir que una hipótesis no funcionó y documentar ese resultado sin sesgarlo para que parezca que funcionó. Los resultados negativos son tan informativos como los positivos. Un equipo que solo documenta los éxitos tiene un sesgo de confirmación que lo lleva a optimizar el sistema en la dirección equivocada.

La siguiente sección aborda el problema que aparece cuando la optimización no basta: cómo detectar que el sistema está degradándose de forma sistemática y qué tipos de respuesta corresponden a distintos niveles de severidad de degradación.

Capítulo 13 — Observabilidad, Evaluación y Optimización

Sección 07: Detección de degradación y deriva

Un sistema de IA puede degradarse sin que nadie lo note. La degradación no siempre es abrupta —una falla técnica que genera alertas inmediatas— sino frecuentemente gradual: la calidad de las respuestas se deteriora lentamente durante semanas, la tasa de satisfacción del usuario baja dos puntos por mes, el sistema tarda un poco más de lo que tardaba hace tres meses. Si no se tienen las métricas correctas y los mecanismos de detección apropiados, la degradación solo se descubre cuando el problema es suficientemente grave como para que llegue por otras vías: quejas de usuarios, reportes del equipo de negocio, caída en métricas de negocio.

Esta sección describe los tres tipos de degradación que afectan a los sistemas de IA en producción, los mecanismos para detectarlos y el framework de alertas que determina qué acción corresponde a cada nivel de severidad.

Los tres tipos de degradación

Deriva del modelo (model drift). El comportamiento del modelo cambia sin que el sistema haya sido modificado. Esto ocurre cuando el proveedor del modelo actualiza silenciosamente su versión de producción —una práctica común en APIs de LLMs comerciales—, cuando las restricciones de alineación del modelo se ajustan, o cuando la distribución de entrenamiento del modelo se actualiza de formas que no están documentadas en el changelog del proveedor.

La deriva del modelo puede manifestarse como un cambio en el tono de las respuestas (de directo a más cauteloso, o viceversa), un cambio en la longitud promedio de las respuestas, un cambio en la frecuencia con que el modelo se niega a responder ciertas categorías de consultas, o un cambio en la capacidad de seguir instrucciones complejas del system prompt.

La detección de model drift requiere ejecutar el golden set periódicamente y comparar los resultados con la línea base. Si el mismo conjunto de casos que antes producía scores de 0.90

ahora produce scores de 0.82, hay evidencia de deriva del modelo que debe investigarse — incluyendo verificar con el proveedor si hubo una actualización del modelo.

Deriva del contexto (context drift). El conocimiento que alimenta el sistema se vuelve desactualizado. Las políticas cambian pero los documentos en la base vectorial no se actualizan. Los productos se discontinúan pero sus fichas técnicas siguen siendo recuperadas. Las regulaciones evolucionan pero la documentación legal indexada refleja la versión anterior.

La deriva del contexto es el tipo de degradación más frecuente y más silencioso. Es silencioso porque el sistema técnicamente sigue funcionando: recupera documentos, construye contextos, genera respuestas coherentes. El problema es que esas respuestas describen un mundo que ya no existe.

La detección de context drift requiere monitorear la antigüedad del conocimiento que el sistema está usando. Si el 30% de los documentos recuperados tienen más de seis meses de antigüedad en un dominio donde el conocimiento cambia mensualmente, hay un problema de vigencia que debe atenderse. También se puede detectar mediante evaluación humana periódica de muestras de respuestas, comparándolas con las fuentes autorizadas actuales.

Deriva de los datos (data drift). Los usuarios cambian la forma en que interactúan con el sistema. Las consultas que recibe el sistema en el mes 12 de operación son cualitativamente diferentes de las del mes 1: los usuarios más casuales se fueron, quedaron los más sofisticados; el dominio de uso se expandió a áreas que el sistema no fue diseñado para cubrir; eventos externos —una campaña de marketing, un incidente de producto, un cambio regulatorio— generan un pico de consultas sobre temas específicos que el sistema no tiene bien cubiertos.

La data drift se detecta analizando la distribución de consultas en el tiempo. Si la distribución semántica de las consultas de este mes es significativamente diferente de la del mes anterior —medida por técnicas como embedding drift detection—, hay un cambio en el uso del sistema que puede requerir adaptaciones en la arquitectura del contexto.

Framework de alertas por nivel de severidad

No todas las señales de degradación requieren la misma respuesta. Un sistema de alertas bien diseñado tiene tres niveles de severidad con acciones distintas.

Nivel 1 — Logging y análisis posterior. Señales de degradación leve que deben registrarse para análisis, pero que no requieren acción inmediata porque pueden ser

fluctuaciones normales o problemas menores que el equipo puede abordar en el ciclo regular de optimización.

Ejemplos de señales de Nivel 1:

- Groundedness cae 0.03 puntos en la semana, dentro del rango histórico de variabilidad
- Un tipo específico de consulta tiene latencia p95 10% más alta que la semana anterior
- La distribución de consultas muestra una nueva categoría emergente que representa el 5% del tráfico

Acción: registrar automáticamente en el log de observabilidad, revisar en la reunión semanal de operaciones, agregar al backlog de optimización si se confirma como tendencia.

Nivel 2 — Revisión activa. Señales de degradación moderada que requieren investigación activa en las próximas 24-48 horas. El sistema sigue operativo pero hay evidencia de un problema que puede agravarse si no se atiende.

Ejemplos de señales de Nivel 2:

- Groundedness cae más de 0.10 puntos en la semana, consistentemente en múltiples días
- La tasa de satisfacción del usuario cae más de 10 puntos en dos semanas consecutivas
- El costo por solicitud aumenta más de 30% sin aumento proporcional en el tráfico
- El golden set muestra una caída de calidad de más de 0.08 puntos comparado con la semana anterior
- El número promedio de pasos por flujo agentivo aumenta más de 40% en 48 horas

Acción: notificación al equipo de ingeniería de IA, asignación de un responsable de investigación, diagnóstico utilizando las trazas de las solicitudes con peores métricas, plan de intervención con plazo definido.

Nivel 3 — Acción inmediata o rollback. Señales de degradación severa que requieren acción inmediata porque el sistema está causando un impacto negativo significativo en los usuarios o en la organización.

Ejemplos de señales de Nivel 3:

- La tasa de errores técnicos supera el 5% del tráfico
- El groundedness cae por debajo de 0.60 en promedio (el sistema está alucinando consistentemente)
- Se detectan respuestas que contradicen activamente políticas corporativas críticas
- El costo por solicitud se quintuplica de forma inexplicable

- El sistema está en bucle: más del 15% de los flujos agentivos alcanzan el límite de pasos
- El golden set muestra una caída de calidad de más de 0.20 puntos respecto a la línea base

Acción: escalación inmediata, evaluación de rollback a la versión anterior, posible desactivación temporal del sistema si el daño potencial es alto, notificación al equipo de negocio.

FRAMEWORK DE ALERTAS — NIVELES DE RESPUESTA

NIVEL 1 — LOGGING

Señal: degradación dentro de la variabilidad histórica

Acción: registrar → revisar en ciclo semanal → backlog

Tiempo de respuesta: próximo ciclo regular (≤ 7 días)

↓ si persiste o se agrava

NIVEL 2 — REVISIÓN ACTIVA

Señal: degradación significativa y sostenida

Acción: asignar responsable → diagnosticar → intervenir

Tiempo de respuesta: 24-48 horas

↓ si es crítica o impacto alto

NIVEL 3 — ACCIÓN INMEDIATA

Señal: degradación severa con impacto en usuarios/negocio

Acción: escalar → evaluar rollback → notificar negocio

Tiempo de respuesta: horas

Detección de alucinaciones en producción

La detección automática de alucinaciones es uno de los problemas más difíciles de la operación de sistemas de IA, y merece un tratamiento específico. Una alucinación es una afirmación que el modelo produce con confianza pero que no está soportada por el conocimiento disponible en su contexto. En sistemas de IA de producción, las alucinaciones pueden tener consecuencias prácticas graves: un asistente de soporte que cita una política de

garantía que no existe, un agente que reporta datos financieros incorrectos, un sistema legal que menciona jurisprudencia que no corresponde.

Las técnicas de detección automática de alucinaciones en producción se dividen en dos categorías.

Detección por contraste con el contexto. El sistema verifica que las afirmaciones de la respuesta pueden rastrearse hasta fragmentos del contexto que el modelo recibió. Esta es la base de la métrica de groundedness. Si la respuesta contiene una afirmación que no puede vincularse con ningún fragmento del contexto recuperado, la respuesta tiene una posible alucinación. La limitación de esta técnica es que no detecta alucinaciones que son consistentes con el contexto pero incorrectas en el mundo real —si el contexto contiene un documento desactualizado, la respuesta puede ser "groundada" e incorrecta al mismo tiempo—.

Detección por verificación cruzada. Para afirmaciones de alto riesgo —precios, fechas, nombres, cifras—, el sistema puede verificar automáticamente que las afirmaciones de la respuesta coinciden con la fuente autoritativa de esos datos. Por ejemplo, un sistema que genera respuestas sobre precios puede verificar que los precios mencionados en la respuesta coincidan con los que están en la base de datos de productos en ese momento.

Ninguna técnica detecta todas las alucinaciones. El enfoque práctico en producción es: (1) maximizar la groundedness del sistema mediante una buena arquitectura de contexto, (2) implementar detección automática para las categorías de afirmaciones de mayor riesgo, y (3) incluir en el proceso de evaluación humana una revisión periódica específica de respuestas en dominios de alto riesgo.

Monitoreo de tendencias a largo plazo

Además de las alertas de corto plazo, el equipo de operaciones debe mantener una vista de tendencias a largo plazo: cómo han evolucionado las métricas del sistema durante los últimos tres meses, seis meses, un año. Esta vista permite detectar degradaciones muy graduales que no disparan ninguna alerta individual pero que, vistas en perspectiva, representan un deterioro significativo.

Una forma práctica de implementarlo es un informe mensual que compara las métricas del mes actual contra el mismo mes del año anterior y contra el mes en que el sistema se lanzó a producción. Si el sistema era más rápido, más barato y producía mejores scores de calidad en su primer mes de producción que en el mes 12, hay un problema de degradación acumulada que los sistemas de alerta corta no capturaron.

Nota del arquitecto

El mayor riesgo de la degradación silenciosa es que el equipo se adapta a ella. Si el sistema va degradándose un punto de calidad por semana, los usuarios y el equipo de operaciones normalizan cada nuevo nivel como "el estado actual del sistema". Al cabo de tres meses, el sistema tiene una calidad significativamente peor que al inicio, pero nadie lo percibe porque la degradación fue gradual.

El antídoto es mantener siempre una referencia absoluta: la calidad del sistema en su mejor momento documentado. La comparación relevante no es "¿está peor que la semana pasada?" sino "¿está peor que cuando el sistema funcionaba mejor?". El golden set, ejecutado periódicamente con comparación contra la línea base de lanzamiento, provee esa referencia absoluta.

La siguiente sección examina cómo organizar toda esta información de observabilidad en dashboards de operación que permitan al equipo mantener el sistema bajo control en producción continua.

Capítulo 13 — Observabilidad, Evaluación y Optimización

Sección 08: Dashboards y operación en producción

Un sistema de IA en producción necesita un equipo que pueda responder en tiempo razonable cuando algo falla. Ese equipo no puede revisar logs crudos ni leer trazas individuales a escala. Necesita una vista condensada, actualizada y accionable del estado del sistema: un dashboard que comunique en segundos si el sistema está funcionando bien, y que señale con precisión dónde está el problema cuando no lo está.

Esta sección describe cómo diseñar los dashboards de operación para sistemas de IA, qué vistas son necesarias, cómo estructurar las alertas y cómo definir los playbooks de respuesta que el equipo ejecuta cuando una alerta se dispara.

Los tres dashboards de un sistema de IA en producción

No existe un único dashboard que sirva para todos los propósitos. La información que necesita el ingeniero de guardia respondiendo un incidente a las 2am es diferente de la que necesita el gestor de producto revisando la evolución del sistema semana a semana. Hay tres dashboards que corresponden a tres audiencias y propósitos diferentes.

Dashboard de guardia: visión en tiempo real. Diseñado para detección rápida de incidentes. Muestra las métricas que pueden indicar que algo está fallando ahora mismo. La frecuencia de actualización es de segundos a minutos. El objetivo es que el ingeniero de guardia pueda determinar en menos de 60 segundos si hay un incidente activo y de qué tipo.

Elementos del dashboard de guardia:

- Tasa de solicitudes por minuto (con alerta si cae o sube abruptamente)
- Tasa de errores técnicos en tiempo real (con umbral de alerta visual)
- Latencia p95 de los últimos 5 minutos (con comparación contra la media histórica)
- Costo por minuto (para detectar anomalías de gasto)
- Estado de las dependencias críticas: APIs de proveedores de modelos, bases vectoriales,

herramientas del agente

- Últimos incidentes abiertos con estado y responsable asignado

Dashboard operacional: visión diaria y semanal. Diseñado para el equipo de operaciones en su revisión regular. Muestra las métricas de las últimas 24 horas y la última semana, con comparaciones de tendencia. El objetivo es identificar deterioros que no generaron alertas inmediatas pero que son visibles al comparar períodos.

Elementos del dashboard operacional:

- Evolución de groundedness, relevancia y satisfacción del usuario (últimos 7 días)
- Distribución de tipos de consultas y cómo ha cambiado respecto a la semana anterior
- Top 10 de consultas con peor score de calidad (para diagnóstico de áreas débiles)
- Resultados de la ejecución periódica del golden set
- Evolución del costo por solicitud y token usage
- Casos pendientes de revisión humana

Dashboard estratégico: visión mensual. Diseñado para la dirección técnica y el equipo de negocio. Muestra la evolución del sistema en el período y el valor de negocio generado. El objetivo es responder: ¿el sistema está mejorando? ¿está generando el valor que se esperaba?

Elementos del dashboard estratégico:

- Tendencias de calidad del último mes versus el mes anterior y versus el lanzamiento
- Satisfacción del usuario y tasa de escalación a agentes humanos
- Costo total del sistema y costo por usuario activo (para ROI)
- Incidentes del mes: cuántos, de qué tipo, cuánto tiempo de resolución
- Experimentos completados y sus resultados (qué se mejoró, en qué porcentaje)

Diseño de alertas efectivas

Las alertas son solo útiles si el equipo las toma en serio. Si el sistema genera demasiadas alertas con poca severidad, el equipo cae en la "fatiga de alertas": las alertas se vuelven ruido de fondo que se ignora, y cuando aparece una alerta crítica, no recibe la atención que merece.

Los principios del diseño de alertas efectivas para sistemas de IA:

Cada alerta debe tener un dueño. Si la alerta de groundedness bajo se dispara, debe haber una persona o un equipo específico que la recibe, con la responsabilidad explícita de investigarla. Las alertas sin dueño no se investigan.

Las alertas deben incluir contexto de diagnóstico. La alerta no debe ser solo "groundedness cayó por debajo de 0.75". Debe incluir: en qué tipos de consultas cayó, en qué horario, si coincide con algún cambio reciente en el sistema, y un enlace directo a los casos de peor score para inspección.

Las alertas deben tener umbral de resolución, no solo de disparo. Un sistema que alerta cuando una métrica cruza un umbral y no alerta cuando regresa al rango normal llena la bandeja de entrada sin cerrar el ciclo. Cada alerta debe resolverse explícitamente — investigada, intervenida, descartada— no simplemente ignorarse cuando la métrica mejora.

El número de alertas activas simultáneas debe ser manejable. Si el equipo tiene más de cinco alertas activas al mismo tiempo, el sistema de alertas está mal calibrado. Las alertas de Nivel 1 deben ser excepcionales, no el estado normal del sistema.

Playbooks de respuesta operacional

Un playbook es un protocolo documentado que el equipo sigue cuando ocurre un evento específico. Los playbooks son la diferencia entre una respuesta coordinada y eficiente y una respuesta improvisada que tarda el doble y llega a conclusiones incorrectas.

Los sistemas de IA en producción deben tener playbooks para al menos los siguientes eventos:

Playbook: caída de calidad (groundedness o relevancia bajo umbral)

1. Verificar si coincide con un cambio reciente en el sistema (despliegue de nueva versión, actualización de base vectorial, cambio de proveedor)
2. Ejecutar el golden set para confirmar si es un problema sistémico o localizado
3. Identificar los tipos de consultas más afectados usando el dashboard operacional
4. Recuperar las trazas de los casos con peor score para diagnóstico de contexto
5. Formular hipótesis (¿documentos desactualizados? ¿system prompt inadecuado para esta categoría de consulta? ¿model drift?)
6. Si se confirma la hipótesis, implementar intervención y monitorear
7. Si no se puede diagnosticar en 4 horas, escalar a Nivel 2 con rollback como opción

Playbook: pico de latencia

1. Verificar el estado de las dependencias del sistema (API del proveedor del modelo, base vectorial)
2. Analizar si el aumento coincide con un aumento en el volumen de tráfico
3. Revisar si hubo cambios en la configuración del sistema que aumenten el tamaño del

contexto

4. Verificar si el pico es en el paso de recuperación RAG, en la llamada al modelo, o en el post-procesamiento
5. Si el cuello de botella es la API del proveedor, implementar cache o reducir llamadas concurrentes
6. Si el cuello de botella es el tamaño del contexto, revisar la estrategia de compresión

Playbook: alerta de bucle en agente

1. Identificar el identificador de las solicitudes en bucle usando el monitor de pasos por flujo
2. Inspeccionar las trazas de esas solicitudes para entender en qué paso se produce el bucle
3. Determinar si el bucle es por una herramienta que falla y devuelve un resultado que el agente no sabe interpretar, o por un criterio de terminación mal definido
4. Si el bucle produce costo descontrolado, implementar un límite de emergencia en el número máximo de pasos
5. Diseñar la corrección en el sistema de planificación del agente y probarla en staging
6. Desplegar la corrección y monitorear los primeros 100 flujos posteriores

La documentación de incidentes como base de conocimiento

Cada incidente resuelto debe documentarse en un post-mortem estructurado. El post-mortem no es un documento de culpabilidad; es un registro de conocimiento sobre el comportamiento del sistema que sirve para:

- Identificar si el mismo tipo de incidente se ha repetido (lo que indica que la causa raíz no fue corregida de forma permanente)
- Capacitar a nuevos miembros del equipo con casos reales de diagnóstico
- Alimentar el diseño de nuevas alertas y actualizaciones de playbooks
- Comunicar al equipo de negocio qué ocurrió, cuál fue el impacto y qué se hizo para evitar recurrencia

Un post-mortem efectivo responde: qué ocurrió, cuándo se detectó, cuál fue el impacto, cuál fue la causa raíz, qué se hizo para resolverlo, y qué cambios en el sistema o en los procesos evitan la recurrencia.

Nota del arquitecto

El diseño de dashboards y playbooks es frecuentemente la última tarea de un proyecto de despliegue de IA —la que se hace "cuando haya tiempo" después del lanzamiento. Este orden de prioridades produce equipos que reaccionan a los incidentes de forma ad-hoc durante las

primeras semanas críticas de producción, cuando el riesgo de incidentes es mayor y el equipo tiene menos experiencia con el comportamiento real del sistema.

El momento correcto para diseñar los dashboards y los playbooks es antes del lanzamiento, durante las pruebas en staging, cuando hay tiempo para pensar sin la presión de un incidente activo. Los dashboards y los playbooks son parte del sistema tanto como el código que procesa las solicitudes. No son documentación adicional; son infraestructura operativa.

La siguiente sección examina los patrones de observabilidad que han probado su valor en producción y los anti-patrones que crean ilusión de observabilidad sin la capacidad real de diagnóstico.

Capítulo 13 — Observabilidad, Evaluación y Optimización

Sección 09: Patrones y anti-patrones de observabilidad

La experiencia colectiva de equipos que han operado sistemas de IA en producción durante años ha producido un conjunto de patrones que funcionan de forma consistente y un conjunto de anti-patrones que crean ilusión de observabilidad sin la capacidad real de diagnóstico. Esta sección los cataloga de forma directa, sin rodeos, porque el reconocimiento temprano de los anti-patrones ahorra semanas de trabajo en la dirección equivocada.

Patrones que funcionan

Patrón: Observabilidad como ciudadano de primera clase. Los equipos que obtienen mejores resultados operativos de sus sistemas de IA tratan la observabilidad con el mismo rigor que el código de producción. Esto significa: instrumentación diseñada en la misma sprint que el feature que que instrumenta, tests de la instrumentación (que verifican que los spans se producen correctamente), y code review que incluye revisión de si el cambio tiene la observabilidad adecuada. Cuando la observabilidad es una tarea de segunda clase, se omite bajo presión y el sistema entra en producción opaco.

Patrón: Métricas en capas. Las métricas se organizan en niveles de granularidad: las métricas de sistema (latencia, errores, costo) se ven de forma agregada y continua; las métricas de calidad (groundedness, relevancia) se calculan sobre muestras y se ven en períodos más largos; las métricas de negocio (satisfacción, escalaciones) se revisan mensualmente. Cada capa tiene su propio dashboard y su propia cadencia de revisión. Los equipos que intentan ver todo en el mismo dashboard producen vistas que son útiles para nadie.

Patrón: Trazabilidad por defecto para incidentes. Cuando se abre un incidente, el primer paso del playbook es siempre recuperar las trazas de los casos afectados. Esta es una disciplina, no una opción. Los equipos que no la tienen como primer paso tienden a pasar tiempo formulando hipótesis sin datos, lo que lleva a diagnósticos incorrectos y soluciones

que no resuelven el problema real. La trazabilidad de contexto solo es útil si se usa sistemáticamente.

Patrón: Golden set como referencia permanente. El golden set no se usa solo para las pruebas de lanzamiento; se ejecuta periódicamente y sus resultados se comparan siempre contra la línea base de lanzamiento, no solo contra el período anterior. Esto impide la normalización de la degradación. Un equipo que compara solo con el período anterior puede no notar que el sistema lleva seis meses degradándose gradualmente.

Patrón: Separación entre alertas operativas y alertas de calidad. Las alertas operativas (errores técnicos, latencia extrema, costo inusual) y las alertas de calidad (groundedness bajo, satisfacción del usuario cayendo) se manejan con canales y playbooks diferentes. Las alertas operativas requieren respuesta en minutos y son responsabilidad del equipo de infraestructura. Las alertas de calidad requieren investigación en horas y son responsabilidad del equipo de IA. Mezclarlas en el mismo canal produce confusión sobre quién debe responder y con qué urgencia.

Patrón: Revisión humana estratificada. La evaluación humana se aplica donde tiene mayor valor: a los casos extremos (scores de calidad muy bajos o muy altos para calibración del evaluador automático), a los nuevos patrones de consulta que no estaban en los datos de calibración, y a los dominios de alto riesgo donde las consecuencias de una respuesta incorrecta son graves. No se distribuye uniformemente sobre todo el tráfico, donde la mayoría de los casos son ordinarios y la evaluación humana tiene rendimientos decrecientes.

Anti-patrones que crean ilusión de observabilidad

Anti-patrón: Monitorear solo la API. El equipo implementa monitoreo de la llamada a la API del proveedor del modelo —latencia, errores, costo— y lo llama "observabilidad". Pueden ver si el sistema responde, pero no si responde bien. Las cuatro dimensiones de observabilidad quedan reducidas a una. Los problemas de calidad son invisibles hasta que los usuarios se quejan.

Anti-patrón: Confiar en los scores del LLM-as-judge sin calibración. El equipo configura un pipeline de evaluación automática, ve que el groundedness promedio es 0.87, y asume que el sistema está produciendo respuestas de alta calidad. Nunca calibra el evaluador contra evaluaciones humanas. Seis meses después, cuando una auditoría externa revisa las respuestas del sistema, descubre que el evaluador automático estaba sobreestimando la calidad de manera sistemática por un sesgo en el prompt de evaluación.

Anti-patrón: Dashboard sin umbral. El equipo construye un dashboard con múltiples métricas, pero no define umbrales de alerta ni criterios de interpretación. Cada semana, el equipo mira los números y tiene una discusión subjetiva sobre si el sistema está funcionando bien. Sin umbrales, los deterioros graduales no se detectan hasta que son obvios para todos. Sin criterios de interpretación, el equipo no sabe cuándo actuar.

Anti-patrón: Instrumentación retroactiva bajo presión. Cuando ocurre un incidente serio, el equipo descubre que no tiene la instrumentación para diagnosticarlo. Se instrumenta el sistema de urgencia durante el incidente, bajo presión, con código que no está bien testeado y que puede introducir nuevos problemas. La instrumentación retroactiva de emergencia es siempre más cara, más lenta y de peor calidad que la instrumentación diseñada antes del lanzamiento.

Anti-patrón: Alertas sin dueño. El equipo configura decenas de alertas, pero no asigna un dueño a cada tipo de alerta. Cuando las alertas se disparan, hay una pausa de varios minutos (o más) mientras el equipo determina quién debe responder. En sistemas donde los usuarios están siendo afectados activamente, esa pausa tiene un costo real.

Anti-patrón: Evaluar la calidad solo con el equipo técnico. El AI Engineer o el equipo de ML evalúa las respuestas del sistema según criterios técnicos de calidad. Pero los criterios de calidad técnica no siempre coinciden con los criterios de calidad del negocio. Una respuesta técnicamente correcta —groundedness 0.95, coherente, bien estructurada— puede ser inapropiada para el tono de la marca o puede omitir información que el usuario de negocio considera esencial. La evaluación de calidad debe incluir al menos periódicamente a representantes del negocio y de los usuarios finales.

Anti-patrón: Optimizar una métrica a expensas de otras. El equipo está bajo presión para mejorar el score de relevancia del sistema. Modifica el sistema de recuperación para privilegiar la relevancia. El score de relevancia sube. La latencia también sube 40%, y la satisfacción del usuario cae porque el sistema ahora tarda demasiado. Este anti-patrón ocurre cuando el proceso de optimización no tiene un criterio de éxito multidimensional y cuando no hay un paso explícito de verificar el impacto en métricas secundarias antes de consolidar un cambio.

Anti-patrón: Golden set estático que nunca se actualiza. El equipo construye un golden set al momento del lanzamiento y lo usa indefinidamente. A medida que el dominio evoluciona —nuevos productos, nuevas políticas, nuevos tipos de consulta de los usuarios—, el

golden set queda desactualizado. El sistema puede pasar con scores altos en un golden set que ya no representa el uso real del sistema.

La heurística del operador honesto

Una prueba práctica para evaluar si el sistema de observabilidad es adecuado: cuando el sistema tiene un problema de calidad que impacta a los usuarios, ¿el equipo puede diagnosticar la causa raíz sin conjeturas en menos de dos horas? Si la respuesta es no —si el diagnóstico requiere más de dos horas, involucra conjeturas sin datos que las soporten, o requiere agregar instrumentación que no existía— el sistema de observabilidad tiene una brecha que debe corregirse.

Esta heurística no garantiza que el sistema de observabilidad sea perfecto. Garantiza que es funcionalmente suficiente para el propósito que justifica su existencia: mantener el sistema operable con calidad sostenida en producción.

Nota del arquitecto

Los anti-patrones descritos no son errores de equipos incompetentes. Son las respuestas naturales de equipos competentes bajo presiones reales: presión de tiempo que lleva a dejar la instrumentación para después, presión de costo que lleva a reducir la evaluación humana, presión de resultados que lleva a optimizar una métrica sin revisar las otras. El AI Engineer que reconoce estos anti-patrones en su propio contexto no necesita juzgarlos; necesita diseñar contra ellos: estructurar el trabajo de forma que la presión de tiempo no pueda eliminar la observabilidad como primera baja.

La siguiente sección integra todos los conceptos del capítulo en un caso de estudio empresarial concreto: cómo una organización implementó observabilidad completa para un sistema de IA de soporte al cliente y qué resultados obtuvo.

Capítulo 13 — Observabilidad, Evaluación y Optimización

Sección 10: Caso de estudio empresarial

Una empresa de servicios financieros con 800,000 clientes activos desplegó un asistente virtual de atención al cliente basado en RAG para responder consultas sobre productos de inversión, crédito hipotecario y seguros. El sistema fue diseñado para reducir el volumen de llamadas al contact center —que estaba saturado— y mejorar el tiempo de respuesta a los clientes.

Este caso describe cómo el equipo de IA enfrentó los problemas de producción y cómo la observabilidad fue la diferencia entre resolver esos problemas y perder el proyecto.

El contexto del proyecto

El asistente fue lanzado en producción en el mes 0. La arquitectura inicial era competente: un sistema RAG con una base vectorial que indexaba el catálogo de productos, los manuales de tarifas y las preguntas frecuentes del área de soporte. El system prompt definía el tono de la institución financiera, las restricciones de lo que el asistente podía y no podía hacer, y las instrucciones para escalar a un agente humano cuando la consulta lo requería.

La instrumentación del lanzamiento era mínima: logging de errores técnicos, latencia de la API del modelo y costo por solicitud. No había medición de calidad de respuestas, no había trazabilidad de contexto y no había golden set.

Durante las primeras tres semanas, los indicadores visibles eran positivos: el sistema respondía, los errores técnicos eran inferiores al 0.5%, la latencia era aceptable. El equipo estaba satisfecho.

El problema que no se veía

En la semana cuatro, el equipo de negocio reportó dos hallazgos preocupantes. Primero, la tasa de escalación a agentes humanos no había disminuido; de hecho, había aumentado ligeramente respecto a antes del lanzamiento. Segundo, el equipo de atención al cliente

recibía más quejas que de costumbre sobre información incorrecta que los clientes decían haber obtenido del asistente virtual.

El equipo de IA revisó los logs técnicos y no encontró nada inusual. El sistema estaba funcionando correctamente desde una perspectiva técnica. No había forma de determinar qué respuestas específicas habían sido incorrectas ni por qué el sistema las había producido.

La discusión entre el equipo técnico y el equipo de negocio fue improductiva durante dos semanas. El equipo técnico argumentaba que los logs no mostraban problemas. El equipo de negocio argumentaba que los clientes estaban recibiendo información incorrecta. Nadie podía resolver la discusión con datos.

La intervención de observabilidad

El equipo de IA tomó la decisión de parar el roadmap de nuevas funcionalidades durante dos semanas y dedicar ese tiempo a implementar observabilidad completa. Fue una decisión difícil de defender porque el sistema técnicamente funcionaba.

La implementación cubrió las cuatro dimensiones:

Trazabilidad de contexto. Se agregó instrumentación para registrar, en cada solicitud: los documentos recuperados con sus metadatos de fecha de actualización, el score de relevancia de cada documento recuperado, la versión del system prompt activa, y el número de tokens de cada componente del contexto.

Evaluación automática de calidad. Se implementó un pipeline de LLM-as-judge que evaluaba groundedness y relevancia en una muestra del 15% del tráfico. El pipeline tomaba la consulta del usuario, el contexto recuperado y la respuesta generada, y producía scores en ambas dimensiones.

Golden set. Se construyó un golden set de 200 casos con participación del equipo de negocio y del equipo de atención al cliente, que identificaron los tipos de consultas más frecuentes y las respuestas esperadas para cada una. Se ejecutó el golden set como prueba de lanzamiento de la observabilidad.

Dashboard operacional. Se construyó un dashboard semanal que mostraba las tendencias de groundedness, relevancia y satisfacción del usuario, con desglose por tipo de consulta.

Los hallazgos de la observabilidad

Los resultados de la primera semana de observabilidad completa revelaron el problema con precisión.

El groundedness promedio del sistema era de 0.91 para las consultas sobre preguntas frecuentes y tarjetas de crédito. Pero para las consultas sobre créditos hipotecarios, el groundedness era de 0.67. El sistema producía respuestas sobre hipotecas que no podían rastrearse hasta los documentos del contexto recuperado.

La revisión de las trazas de las solicitudes hipotecarias reveló la causa raíz: el catálogo de productos hipotecarios había sido actualizado hace seis semanas con nuevas tasas de interés y condiciones de financiamiento. El documento actualizado existía en el servidor documental de la empresa, pero no había sido reindexado en la base vectorial del sistema RAG. El sistema seguía recuperando el documento de la versión anterior, con las tasas de interés antiguas.

Cuando el sistema respondía preguntas sobre hipotecas, citaba tasas que ya no estaban vigentes. Los clientes que llamaban al contact center para contratar un hipoteca recibían condiciones diferentes de las que el asistente había prometido. De ahí las quejas y las escalaciones.

El tiempo entre la identificación de la causa raíz y la solución fue de cuatro horas: reindexar el documento actualizado, verificar que el sistema estaba recuperando la versión correcta, ejecutar el golden set para confirmar que el groundedness de las consultas hipotecarias había subido al nivel esperado.

La mejora del proceso

El hallazgo tuvo una consecuencia adicional más valiosa que la corrección inmediata: reveló que el proceso de actualización de la base vectorial no estaba integrado con el proceso de actualización de documentos. Cuando el equipo de productos actualizaba un documento, no había ningún mecanismo que notificara al equipo de IA que el documento debía ser reindexado.

El equipo diseñó un proceso de actualización automática: cuando un documento en el sistema de gestión documental era modificado y marcado como aprobado, un webhook disparaba automáticamente la reindexación del documento en la base vectorial. Si la reindexación fallaba, se generaba una alerta. El proceso eliminó la clase de error que había causado el problema hipotecario.

Los resultados cuatro meses después

Con observabilidad completa y el proceso de actualización corregido, los indicadores del sistema cambiaron de forma medible.

La tasa de escalación a agentes humanos disminuyó un 28% respecto al nivel previo al lanzamiento. Las quejas sobre información incorrecta cayeron a casi cero en las primeras semanas después de la corrección del proceso. El groundedness promedio del sistema se estabilizó en 0.89 en todos los tipos de consulta.

El equipo pudo, por primera vez, responder con datos la pregunta que el equipo de negocio hacía: ¿el asistente está funcionando bien? Y pudo identificar con precisión los dos tipos de consultas que seguían teniendo calidad más baja —consultas sobre seguros de vida y sobre inversiones en fondos de renta variable— para enfocar el siguiente ciclo de optimización en esas áreas.

Lecciones del caso

Primera lección. La ausencia de incidentes técnicos no es evidencia de que el sistema funciona bien. El sistema de esta empresa funcionaba técnicamente durante cuatro semanas mientras producía información incorrecta que causaba impacto real a los clientes. Las métricas técnicas y las métricas de calidad son dimensiones distintas.

Segunda lección. Sin trazabilidad, el tiempo de diagnóstico se mide en semanas y depende de conjeturas. Con trazabilidad, el diagnóstico se mide en horas y se basa en evidencia. La diferencia no es marginal; es la diferencia entre un equipo que puede operar el sistema y uno que no puede.

Tercera lección. La observabilidad no es solo un instrumento de diagnóstico reactivo. Es un instrumento de mejora continua. Sin los datos que la observabilidad produce, el equipo no sabe qué tipos de consultas mejorar, qué documentos están mal indexados ni qué cambios en el sistema tienen impacto positivo. Con esos datos, el equipo puede operar con intención en lugar de con intuición.

Cuarta lección. El costo de implementar observabilidad antes del lanzamiento habría sido significativamente menor que el costo de las dos semanas de parálisis del equipo, las quejas de clientes, la pérdida de confianza del equipo de negocio y la implementación de urgencia que finalmente ocurrió. La observabilidad como inversión inicial tiene retorno positivo en casi cualquier sistema de IA en producción.

Nota del arquitecto

El caso descrito es representativo de un patrón que se repite con variaciones en la mayoría de las implementaciones de IA empresarial que no tienen observabilidad desde el inicio. Los detalles cambian —el dominio, el tipo de falla, el impacto en el negocio— pero la estructura es la misma: el sistema funciona técnicamente, el impacto en los usuarios es invisible hasta que llega por canales de negocio, el diagnóstico es imposible sin instrumentación, el equipo pierde tiempo y credibilidad.

La observabilidad no evita todos los problemas. Pero hace que los problemas sean detectables, diagnosticables y resolvibles. En un sistema de producción que opera durante meses o años, esa capacidad es el diferenciador entre un sistema que mejora con el tiempo y uno que se deteriora sin que nadie lo note.

La siguiente sección traduce todos estos conceptos en un ejercicio práctico: el laboratorio donde el lector define su propio framework de observabilidad para un caso de uso específico.

Capítulo 13 — Observabilidad, Evaluación y Optimización

Sección 11: Laboratorio práctico

Este laboratorio tiene un objetivo específico: diseñar un framework de observabilidad completo para un caso de uso concreto, antes de que ese sistema entre en producción. El ejercicio no requiere código. Requiere tomar decisiones de diseño —qué se mide, cómo se mide, qué umbrales se usan, qué acciones corresponden a cada umbral— que son las decisiones que determinan si el sistema será operable o no.

El caso de uso

Se va a diseñar la observabilidad para el siguiente sistema:

Asistente de soporte técnico para una empresa de software SaaS. El asistente ayuda a los usuarios de la plataforma a resolver problemas de configuración, responder dudas sobre funcionalidades y guiar flujos de trabajo dentro del producto. Está basado en RAG con acceso a la documentación del producto, artículos de la base de conocimiento y guías de inicio rápido. Los usuarios interactúan mediante chat en la propia plataforma. El sistema recibe aproximadamente 3,000 consultas por día en horario laboral (8am-8pm). Una respuesta incorrecta o incompleta puede llevar al usuario a realizar configuraciones erróneas que afectan sus datos o flujos de trabajo.

Parte 1: Definición de métricas

El primer paso es definir qué se mide y por qué. Para este caso de uso específico, el estudiante debe seleccionar y justificar cinco métricas de calidad y tres métricas operacionales.

Tarea 1.1: Métricas operacionales (completar la tabla)

Métrica	Por qué es relevante para este caso	Cómo se calcula	Umbral de alerta propuesto
(métrica 1)			
(métrica 2)			
(métrica 3)			

Consideraciones: el sistema opera en horario laboral con 3,000 consultas diarias, lo que equivale a aproximadamente 250 consultas por hora en pico. Los usuarios son profesionales que usan el producto para trabajo real. La latencia que consideran aceptable es diferente a la de un chatbot de entretenimiento.

Tarea 1.2: Métricas de calidad (completar la tabla)

Métrica	Definición para este caso	Cómo se mide	Umbral de alerta propuesto
(métrica 1)			
(métrica 2)			
(métrica 3)			
(métrica 4)			
(métrica 5)			

Consideraciones: una respuesta incorrecta sobre configuración puede llevar a errores que el usuario debe deshacer. El costo de una respuesta incorrecta es más alto que el de una respuesta incompleta que lleva al usuario a preguntar de nuevo.

Parte 2: Diseño del golden set

El estudiante debe diseñar la estructura del golden set para este sistema: qué categorías de casos debe incluir, cuántos casos por categoría, y quiénes deben participar en su construcción.

Tarea 2.1: Categorías del golden set

Identifica al menos seis categorías de casos que el golden set debe cubrir para ser representativo del uso real del sistema:

1. (categoría 1 — descripción y ejemplos de consultas típicas)
2. (categoría 2)
3. (categoría 3)

4. (categoría 4)
5. (categoría 5)
6. (categoría 6)

Tarea 2.2: Casos de borde y adversariales

Identifica al menos tres tipos de casos de borde o adversariales que el golden set debe incluir:

1. (caso borde 1 — descripción y por qué es importante)
2. (caso borde 2)
3. (caso borde 3)

Tarea 2.3: Proceso de construcción

Describe brevemente:

- Quiénes deben participar en la construcción del golden set (¿solo el equipo técnico? ¿también miembros del equipo de soporte? ¿usuarios reales?)
- Con qué frecuencia debe actualizarse y bajo qué condiciones
- Cómo se determina que una respuesta en el golden set es "correcta"

Parte 3: Framework de alertas

Diseña el framework de alertas para este sistema con tres niveles de severidad. Para cada nivel, define:

- Qué condiciones lo disparan (con los valores numéricos de los umbrales)
- Qué acción debe tomar el equipo
- En qué tiempo debe ocurrir la respuesta
- Quién es responsable de responder

Nivel 1 — Logging:

- Condiciones: (lista de métricas y umbrales)
- Acción:
- Tiempo de respuesta:
- Responsable:

Nivel 2 — Revisión activa:

- Condiciones:
- Acción:

- Tiempo de respuesta:
- Responsable:

Nivel 3 — Acción inmediata:

- Condiciones:
- Acción:
- Tiempo de respuesta:
- Responsable:

Parte 4: Playbook de un incidente específico

El equipo recibe el siguiente reporte del equipo de soporte:

"Varios usuarios están reportando que el asistente les da instrucciones de configuración que no coinciden con lo que ven en la interfaz. Los pasos que describe el asistente no existen en la versión actual del producto."

Escribe el playbook de respuesta para este incidente específico, siguiendo la estructura:

1. **Primer paso: verificación** — ¿Qué datos verificas primero para confirmar el problema? ¿Qué herramienta o sistema usas?
2. **Identificación del alcance** — ¿Cómo determinas si el problema afecta a todas las consultas de configuración o solo a un tipo específico? ¿Qué datos necesitas?
3. **Diagnóstico de causa raíz** — ¿Cuáles son las tres hipótesis más probables para este tipo de problema? ¿Cómo descartarías o confirmarías cada una con los datos disponibles?
4. **Intervención** — Si la causa raíz es que la documentación en la base vectorial corresponde a una versión anterior del producto, ¿qué pasos concretos sigues para resolver el problema?
5. **Verificación de la solución** — ¿Cómo confirmas que el problema está resuelto? ¿Qué métricas monitorizas durante las siguientes 24 horas?
6. **Documentación** — ¿Qué escribes en el post-mortem? ¿Qué cambios en el proceso propones para evitar la recurrencia?

Parte 5: Diseño del pipeline de evaluación

Diseña el pipeline de evaluación completo para este sistema. El diseño debe responder:

Evaluación automática:

- ¿Qué porcentaje del tráfico se evalúa automáticamente?
- ¿Qué modelo se usa como juez y por qué?
- ¿Cuál es el prompt de evaluación para la dimensión de groundedness? (escribe el prompt completo)
- ¿Con qué frecuencia se ejecuta el golden set?

Evaluación humana:

- ¿Qué casos se envían a evaluación humana?
- ¿Quién evalúa (miembro del equipo técnico, miembro del equipo de soporte, usuario experto)?
- ¿Con qué criterio se determina si una respuesta es correcta para este dominio?
- ¿Con qué frecuencia se calibra el evaluador automático contra las evaluaciones humanas?

Respuestas de referencia

Las siguientes respuestas no son la única solución correcta, pero ilustran el nivel de detalle y el razonamiento esperado.

Métricas operacionales propuestas:

Métrica	Por qué es relevante	Cómo se calcula	Umbral de alerta
Latencia p95	Los usuarios profesionales tienen baja tolerancia a esperas. P95 captura los casos lentos que afectan la experiencia	Tiempo entre solicitud y respuesta completa, percentil 95	> 4 segundos (el doble del p50 esperado de ~2s)
Tasa de errores técnicos	Las interrupciones de servicio durante horario laboral tienen impacto directo en la productividad	Errores / total de solicitudes	> 0.5% (umbral bajo dado el impacto en trabajo real)
Tokens de contexto p95	Un contexto que crece sin control indica problemas de compresión o documentos excesivamente largos recuperados	Tokens totales del prompt, percentil 95	> 75% de la ventana de contexto del modelo

Métricas de calidad propuestas:

Métrica	Definición para este caso	Cómo se mide	Umbral de alerta
Groundedness	Cada instrucción de configuración debe poder rastrearse hasta la documentación actual	LLM-as-judge, muestra del 15%	< 0.85 (más estricto que el promedio; errores de config tienen alto costo)
Exactitud de versión	Las instrucciones corresponden a la versión actual del producto	Evaluación humana periódica + comparación con changelog	< 95% de exactitud en evaluación humana mensual
Relevancia	La respuesta aborda la consulta específica del usuario	LLM-as-judge, mismo 15% de muestra	< 0.82 en promedio semanal
Tasa de reformulación	El usuario reformuló su pregunta (señal de que la primera respuesta no satisfizo)	Análisis del historial de conversación	> 20% de conversaciones con reformulación en 48h
Tasa de escalación	El usuario solicitó hablar con un agente humano después de interactuar con el asistente	Log de eventos de escalación	> 15% del total de conversaciones

Nota al instructor

Este laboratorio puede adaptarse a cualquier caso de uso: un asistente de análisis financiero, un asistente legal, un sistema de recomendación de productos. La estructura del ejercicio — definir métricas, diseñar el golden set, crear el framework de alertas, escribir el playbook, diseñar el pipeline de evaluación— es transferible a cualquier sistema de IA en producción. El valor pedagógico está en forzar las decisiones de diseño antes de que el sistema exista, cuando es más fácil pensar con claridad que después de que el primer incidente de producción ocurre.

Capítulo 13 — Observabilidad, Evaluación y Optimización

Sección 12: Checklist del AI Engineer

Este checklist recoge las verificaciones que el AI Engineer debe completar antes de lanzar un sistema de IA a producción y como parte de la operación regular. Está organizado en seis bloques que corresponden a las dimensiones del capítulo.

Bloque 1: Diseño de la observabilidad (antes del lanzamiento)

- [] Las cuatro dimensiones de observabilidad están diseñadas: inferencia, contexto, calidad y comportamiento
- [] La instrumentación está implementada en el mismo sprint que el feature que instrumenta, no como tarea posterior
- [] Se han definido identificadores de traza que permiten correlacionar todos los eventos de una solicitud a través de las etapas del pipeline
- [] La versión del system prompt se registra en cada traza
- [] Los documentos recuperados se registran con sus metadatos: identificador, fuente, fecha de creación, fecha de modificación, score de relevancia
- [] Las herramientas ejecutadas por el agente se registran con sus parámetros y resultados
- [] La instrumentación tiene tests que verifican que los spans se producen correctamente
- [] Las consideraciones de privacidad están resueltas: qué datos de usuario se registran, durante cuánto tiempo, quién puede acceder

Bloque 2: Métricas y umbrales (antes del lanzamiento)

- [] Las métricas operacionales están definidas: latencia (p50, p95, p99), TTFT, tokens por solicitud, costo por solicitud, tasa de errores técnicos
- [] Las métricas de calidad están definidas y son específicas para el caso de uso: qué significa "relevante" y "correcto" para este dominio

- [] Cada métrica tiene un umbral de alerta definido, justificado con el criterio de negocio del sistema
- [] Cada umbral de alerta tiene una acción asociada: quién investiga, con qué prioridad
- [] Los umbrales de alerta han sido validados contra el comportamiento observado del sistema en staging, no definidos en abstracto
- [] Se ha definido con qué frecuencia se revisarán las métricas (tiempo real, diario, semanal, mensual)

Bloque 3: Golden set y pipeline de evaluación (antes del lanzamiento)

- [] El golden set está construido con participación del equipo de negocio y del dominio
- [] El golden set cubre los tipos de consulta más frecuentes, los casos borde conocidos, y los casos adversariales relevantes para el dominio
- [] El golden set incluye casos que verifican las restricciones críticas del sistema (lo que el sistema nunca debe hacer)
- [] El pipeline de evaluación automática (LLM-as-judge) está implementado y ejecutado sobre una muestra del tráfico
- [] El prompt de evaluación del LLM-as-judge está calibrado contra evaluaciones humanas de referencia
- [] Se ha establecido la cadencia de evaluación humana: quién evalúa, con qué frecuencia, con qué protocolo
- [] El proceso de medición del acuerdo entre evaluadores humanos (inter-rater reliability) está definido

Bloque 4: Alertas y dashboards (antes del lanzamiento)

- [] El dashboard de guardia está disponible: muestra métricas operacionales en tiempo real con alertas visuales
- [] El dashboard operacional está disponible: muestra tendencias de calidad con comparación de períodos
- [] Cada alerta tiene un dueño asignado: una persona o equipo específico que la recibe y es responsable de responderla
- [] Las alertas están separadas por canal: alertas operativas (infraestructura) y alertas de calidad (equipo de IA)
- [] El número de alertas activas configuradas es manejable: el equipo puede atender todas las que se disparan en un día normal

- ☐ El sistema de alertas incluye contexto de diagnóstico en cada notificación, no solo el número de la métrica

Bloque 5: Playbooks de operación (antes del lanzamiento)

- ☐ Existe un playbook para caída de calidad (groundedness o relevancia bajo umbral)
- ☐ Existe un playbook para pico de latencia
- ☐ Existe un playbook para bucle o comportamiento anómalo en agentes (si el sistema tiene componentes agentivos)
- ☐ Existe un playbook para aumento inesperado de costo
- ☐ Existe un playbook para drift del modelo (el sistema empieza a comportarse diferente sin cambios en el código)
- ☐ Los playbooks incluyen criterios explícitos para decidir si se hace rollback
- ☐ Los playbooks han sido revisados por el equipo completo y no solo por el autor
- ☐ Existe una plantilla de post-mortem que se completa después de cada incidente de Nivel 2 o superior

Bloque 6: Operación regular (cadencia continua)

Diariamente:

- ☐ Revisión del dashboard de guardia: sin alertas activas o con alertas en proceso de resolución
- ☐ Verificación de que el pipeline de evaluación automática ejecutó sin errores

Semanalmente:

- ☐ Revisión del dashboard operacional: tendencias de métricas de calidad y operacionales
- ☐ Revisión de los casos con peores scores de la semana: ¿hay un patrón? ¿hay una hipótesis de mejora?
- ☐ Revisión de los incidentes de la semana (si los hubo): ¿el playbook funcionó? ¿necesita actualizarse?
- ☐ Verificación de la antigüedad promedio de los documentos recuperados: ¿hay documentos que debería actualizarse?

Mensualmente:

- ☐ Ejecución del golden set y comparación con la línea base de lanzamiento
- ☐ Calibración del evaluador automático contra evaluaciones humanas
- ☐ Revisión del golden set: ¿sigue siendo representativo del uso actual del sistema?
- ☐ Análisis de tendencias a largo plazo: ¿el sistema está mejorando, estable o deteriorando?

- [] Revisión del costo acumulado del sistema y comparación con el valor de negocio generado
- [] Actualización del plan de optimización del próximo ciclo basada en los datos del mes

Ante cada cambio en el sistema:

- [] El golden set se ejecuta en staging antes de desplegar a producción
- [] Las métricas se comparan entre la versión nueva y la versión actual
- [] El cambio se documenta con la justificación, el resultado esperado y el resultado observado
- [] Se define el período de monitoreo intensivo posterior al cambio y el responsable de ese monitoreo

Criterio de preparación para producción

Un sistema de IA está listo para producción desde la perspectiva de la observabilidad cuando puede responder afirmativamente a las siguientes preguntas:

1. Si el sistema produce una respuesta incorrecta ahora mismo, ¿el equipo puede saberlo en menos de 24 horas?
2. Si el equipo detecta una respuesta incorrecta, ¿puede diagnosticar la causa raíz en menos de 2 horas?
3. Si el sistema se degrada gradualmente durante cuatro semanas, ¿hay un mecanismo que lo detecte antes de que el impacto en los usuarios sea significativo?
4. ¿Hay un responsable definido para cada tipo de alerta que el sistema puede generar?
5. ¿Sabe el equipo qué acción tomar cuando cada tipo de alerta se dispara?

Si alguna de estas preguntas recibe una respuesta negativa, el sistema tiene una brecha de observabilidad que debe corregirse antes del lanzamiento, no después.

Capítulo 13 — Observabilidad, Evaluación y Optimización

Sección 13: Resumen del capítulo

Este capítulo estableció que los sistemas de IA fallan de formas cualitativamente diferentes a los sistemas de software tradicionales: pueden responder correctamente desde una perspectiva técnica mientras producen respuestas incorrectas, irrelevantes o desactualizadas. Esa forma de falla silenciosa hace que el monitoreo técnico convencional sea insuficiente y que la observabilidad sea un dominio propio de la ingeniería de IA.

Las ideas centrales del capítulo

La observabilidad tiene cuatro dimensiones. Ninguna por sí sola es suficiente para entender qué está ocurriendo en un sistema de IA en producción.

La primera dimensión —observabilidad de la inferencia— mide el comportamiento técnico del sistema: latencia, tokens consumidos, costo, errores. Es la más fácil de implementar y la menos informativa sobre la calidad del sistema.

La segunda dimensión —observabilidad del contexto— registra qué información exacta recibió el modelo en cada solicitud: qué documentos recuperó el sistema RAG, qué versión del system prompt estaba activa, qué herramientas ejecutó el agente. Es la dimensión más diferenciadora para el diagnóstico de problemas.

La tercera dimensión —observabilidad de la calidad— mide si las respuestas son buenas: relevantes, fundamentadas en el contexto, factualmente correctas, satisfactorias para el usuario. Requiere técnicas de evaluación propias, tanto automáticas como humanas.

La cuarta dimensión —observabilidad del comportamiento— analiza los patrones de uso del sistema en producción: qué tipos de consultas recibe, cómo esos patrones cambian con el tiempo, qué consultas producen consistentemente resultados insatisfactorios.

Las métricas deben estar vinculadas a acciones. Recolectar métricas sin umbrales de alerta definidos y sin acciones asociadas a cada umbral produce dashboards decorativos, no

capacidad operativa. El diseño de las métricas y el diseño de los procesos de respuesta son inseparables.

La evaluación de calidad combina técnicas automáticas y humanas. El LLM-as-judge permite evaluar a escala una muestra del tráfico de producción. Tiene limitaciones conocidas —sesgo de autopreferencia, sensibilidad al formato del prompt de evaluación— que requieren calibración periódica contra evaluaciones humanas. La evaluación humana es indispensable para la calibración, para los dominios de alto riesgo, y para detectar nuevos patrones de falla que el evaluador automático no captura.

La trazabilidad de contexto es el componente de observabilidad más diferenciador. Registrar el contexto completo que el modelo recibió en cada solicitud —con identificadores que permiten reconstruirlo retroactivamente— reduce el tiempo de diagnóstico de incidentes de semanas a horas. La trazabilidad debe diseñarse antes del lanzamiento; implementarla retroactivamente es más costoso y de peor calidad.

Los sistemas de IA tienen tres tipos de degradación. La deriva del modelo ocurre cuando el proveedor actualiza silenciosamente el modelo. La deriva del contexto ocurre cuando el conocimiento en la base vectorial se desactualiza. La deriva de los datos ocurre cuando los patrones de uso de los usuarios cambian. Cada tipo requiere detección y respuesta diferente.

El framework de alertas tiene tres niveles. El Nivel 1 registra para análisis posterior. El Nivel 2 requiere investigación activa en 24-48 horas. El Nivel 3 requiere acción inmediata o rollback. Cada nivel tiene umbrales definidos y responsables asignados.

La optimización continua usa los datos de producción como fuente. No se puede optimizar bien un sistema de IA solo con experimentos de laboratorio. Los datos de producción revelan las áreas débiles reales, los patrones de uso no anticipados y las oportunidades de mejora que solo son visibles con volumen de solicitudes. El ciclo de observación, hipótesis, intervención y medición es el mecanismo de mejora continua.

Los patrones y anti-patrones son reconocibles. Los equipos que obtienen buenos resultados operativos tratan la observabilidad como ciudadano de primera clase, usan métricas en capas, mantienen golden sets permanentes y separan las alertas operativas de las de calidad. Los equipos que no obtienen buenos resultados monitorean solo la API, confían sin calibración en los scores automáticos, configuran alertas sin umbrales o sin dueños, e instrumentan retroactivamente bajo presión de incidentes.

El principio que atraviesa el capítulo

Hay un principio que unifica todas las secciones: la observabilidad es una propiedad del sistema, no una herramienta que se agrega después. Un sistema que no se diseñó para ser observable no puede serlo retroactivamente sin esfuerzo significativo. La decisión de ser observable debe tomarse en el mismo momento en que se diseña la arquitectura del sistema, con el mismo rigor que las decisiones de funcionalidad.

Este principio tiene una consecuencia práctica para el AI Engineer: la observabilidad no es la tarea de la última semana antes del lanzamiento. Es una dimensión del diseño que aparece en el primer sprint y se mantiene activa durante toda la vida operativa del sistema.

Conexión con el módulo

El capítulo 13 ocupa un lugar específico en la estructura del módulo: después de haber aprendido a construir sistemas de IA complejos —agentes, RAG, coordinación multi-agente, arquitecturas empresariales—, la observabilidad es la respuesta a la pregunta de cómo operar esos sistemas responsablemente. Un sistema que no puede monitorearse, evaluarse ni mejorarse basándose en datos no es un sistema que puede sostenerse en producción.

El capítulo siguiente —Seguridad y Gobernanza— es el complemento natural de la observabilidad. La observabilidad permite ver qué ocurre en el sistema. La seguridad y la gobernanza protegen el sistema de amenazas deliberadas que la observabilidad por sí sola no puede contener.

Lo que el lector debe poder hacer al terminar este capítulo

Al completar el capítulo, el AI Engineer debe ser capaz de:

- Diseñar el plan de observabilidad de un sistema de IA antes de su lanzamiento, cubriendo las cuatro dimensiones
- Definir métricas de calidad específicas para un caso de uso dado, con umbrales de alerta justificados
- Construir un golden set representativo con participación del equipo de dominio
- Implementar un pipeline de evaluación combinado (LLM-as-judge + evaluación humana) con calibración periódica
- Diseñar un sistema de trazabilidad de contexto que permita el diagnóstico retroactivo de incidentes
- Detectar los tres tipos de degradación en producción y definir el nivel de respuesta apropiado para cada señal

- Diseñar dashboards de operación para las tres audiencias: guardia, equipo operacional, dirección
 - Escribir playbooks de respuesta para los incidentes más probables del sistema
 - Ejecutar el ciclo de optimización continua: observar, formular hipótesis, intervenir, medir, documentar
 - Reconocer los anti-patrones de observabilidad y diseñar contra ellos desde el inicio del proyecto
-

Capítulo 13 — Observabilidad, Evaluación y Optimización

Sección 14: Autoevaluación

Las siguientes preguntas permiten verificar la comprensión de los conceptos centrales del capítulo. Para cada pregunta, intenta formular una respuesta antes de consultar la clave de respuestas.

Pregunta 1

Un sistema de IA en producción tiene 99.8% de disponibilidad, latencia p95 de 1.8 segundos y costo por solicitud dentro del presupuesto. El equipo de negocio reporta que la satisfacción de los usuarios cayó 12 puntos en el último mes. ¿Qué dimensión de observabilidad está faltando y qué técnica usarías para diagnosticar el problema?

Pregunta 2

¿Cuál es la diferencia entre monitoreo y observabilidad en el contexto de sistemas de IA? Da un ejemplo concreto de un problema que el monitoreo no detectaría pero la observabilidad sí.

Pregunta 3

Estás implementando un evaluador LLM-as-judge para medir la groundedness de las respuestas de un asistente de soporte técnico. Describe tres limitaciones conocidas de este enfoque y qué medidas tomarías para mitigar cada una.

Pregunta 4

Un sistema de RAG que lleva ocho meses en producción comienza a producir respuestas con información desactualizada sobre los productos de la empresa. El sistema técnico funciona correctamente: latencia normal, sin errores. ¿Qué tipo de degradación es este? ¿Cómo lo detectarías con las herramientas de observabilidad del capítulo?

Pregunta 5

¿Para qué sirve el golden set? ¿Por qué no es suficiente comparar las métricas del sistema solo con las del período anterior (semana a semana)?

Pregunta 6

Un agente en producción está ejecutando en promedio 18 pasos por solicitud cuando fue diseñado para operar en 4-8 pasos. ¿A qué nivel de alerta corresponde esta señal según el framework del capítulo? ¿Cuáles son las hipótesis de causa raíz más probables y cómo las investigarías?

Pregunta 7

¿Cuál es la diferencia entre la trazabilidad de contexto para sistemas de IA y el logging tradicional de software? ¿Por qué el logging de eventos no es suficiente para diagnosticar problemas de calidad en sistemas de IA?

Pregunta 8

Un equipo implementó un dashboard con 15 métricas, todas actualizándose en tiempo real. Hay 8 alertas activas la mayoría de los días. El equipo responde a las alertas más urgentes y el resto se ignoran. ¿Qué anti-patrón describe esta situación? ¿Cómo lo corregirías?

Pregunta 9

Explica el concepto de "deriva del modelo" (model drift). ¿Por qué es particularmente difícil de detectar? ¿Qué mecanismo del capítulo permite detectarlo de forma más fiable?

Pregunta 10

Una empresa quiere implementar A/B testing para comparar dos versiones de su sistema de recuperación RAG: la versión actual (sin re-ranking) y una nueva versión (con re-ranking semántico). La empresa tiene 500 solicitudes por día. ¿Cuáles son los desafíos específicos de este experimento y qué recomendaciones darías para que produzca resultados confiables?

Clave de respuestas

Respuesta 1

La dimensión faltante es la observabilidad de la calidad: no se están midiendo métricas que indiquen si las respuestas son buenas. Las métricas operacionales (disponibilidad, latencia, costo) están en verde porque miden si el sistema responde, no si responde bien.

Para diagnosticar el problema: implementar un pipeline de evaluación LLM-as-judge para medir groundedness y relevancia sobre una muestra del tráfico reciente. Construir un golden set con el equipo de negocio y ejecutarlo para comparar con el comportamiento esperado. Revisar las trazas de contexto de las solicitudes de los últimos 30 días para detectar si los documentos recuperados tienen fechas de modificación antiguas (posible deriva del contexto).

Respuesta 2

El monitoreo recolecta métricas predefinidas sobre comportamientos conocidos. La observabilidad es la capacidad de inferir el estado interno del sistema a partir de sus salidas, incluyendo comportamientos no previstos en el diseño.

Ejemplo: el monitoreo puede detectar que el sistema devuelve HTTP 200 con latencia aceptable. No puede detectar que el sistema está respondiendo preguntas sobre precios con tarifas desactualizadas porque el documento de tarifas en la base vectorial tiene seis meses de antigüedad. La observabilidad —concretamente la trazabilidad de contexto con metadatos de fecha de los documentos recuperados— sí puede detectarlo.

Respuesta 3

Primera limitación: sesgo de autopreferencia. Si el evaluador y el sistema evaluado son del mismo modelo o familia de modelos, el evaluador tiende a calificar más alto las respuestas de ese estilo. Mitigación: usar un modelo evaluador diferente al modelo del sistema, o calibrar el evaluador contra evaluaciones humanas para detectar y corregir el sesgo.

Segunda limitación: incapacidad para detectar alucinaciones sobre conocimiento externo al contexto. El evaluador puede confirmar que la respuesta es consistente con el contexto recuperado sin detectar que el contexto recuperado contiene información desactualizada. Mitigación: complementar con verificación cruzada contra fuentes autorizadas para las categorías de afirmaciones de mayor riesgo.

Tercera limitación: sensibilidad al formato del prompt de evaluación. Cambios menores en el prompt del evaluador pueden producir scores significativamente diferentes. Mitigación: establecer un prompt de evaluación canónico, versionarlo junto con el sistema, y validar cualquier cambio al prompt contra la calibración humana de referencia antes de adoptarlo.

Respuesta 4

Es deriva del contexto (context drift): el conocimiento en la base vectorial se desactualizó porque los documentos de productos fueron actualizados en su fuente original pero no fueron reindexados en el sistema RAG.

Detección: la trazabilidad de contexto con metadatos de fecha de los documentos recuperados habría mostrado que los documentos con mayor frecuencia de recuperación tienen fechas de modificación de hace varios meses, cuando la empresa actualiza sus productos regularmente. El golden set ejecutado periódicamente habría mostrado una caída en el score de exactitud para las consultas sobre productos actualizados.

Respuesta 5

El golden set sirve para tener una referencia absoluta de calidad: un conjunto de casos con respuestas correctas conocidas que se ejecuta periódicamente y cuyos resultados se comparan siempre contra la línea base de lanzamiento, no solo contra el período anterior.

Si se comparan las métricas solo semana a semana, el sistema puede degradarse gradualmente sin que ninguna comparación individual muestre una diferencia significativa. Cada semana, el nuevo nivel (ligeramente peor) se convierte en la referencia para la siguiente comparación. Al cabo de seis meses, el sistema puede estar significativamente peor que en el lanzamiento sin que ninguna comparación semanal haya generado una alerta. El golden set, al comparar siempre contra la línea base absoluta, detecta esta degradación acumulada.

Respuesta 6

Corresponde a una alerta de Nivel 2 (revisión activa en 24-48 horas): el número de pasos es más del doble del diseñado, lo que indica un problema sistémico que requiere investigación activa pero no necesariamente acción inmediata si el sistema sigue produciendo respuestas y el costo es manejable. Si el costo proyectado o el tiempo de respuesta se vuelven inaceptables, escalaría a Nivel 3.

Hipótesis de causa raíz: (1) una herramienta del agente está fallando silenciosamente y devolviendo resultados vacíos o erróneos, lo que lleva al agente a repetir la llamada con parámetros ligeramente diferentes; (2) el criterio de terminación del ciclo agentivo no es suficientemente preciso para los tipos de consulta que el sistema está recibiendo actualmente; (3) hubo un cambio en el sistema de planificación que amplió el espacio de acciones exploradas.

Para investigar: revisar las trazas de los flujos con más de 15 pasos, identificar qué herramientas se repiten, verificar los resultados de esas herramientas, comparar la distribución de tipos de consulta actual con la histórica para detectar si hay nuevos tipos de consulta que el agente no sabe resolver eficientemente.

Respuesta 7

El logging de eventos registra qué hizo el sistema: "el sistema llamó a la API del modelo y recibió una respuesta". La trazabilidad de contexto registra con qué información lo hizo: qué documentos tenía disponibles, qué versión del system prompt estaba activa, qué herramientas ejecutó el agente.

La diferencia es crítica para el diagnóstico de problemas de calidad. Si el sistema produce una respuesta incorrecta, el log de eventos dice que el sistema respondió; la traza de contexto dice qué tenía disponible para generar esa respuesta. El diagnóstico de calidad requiere la segunda información, no la primera. El logging solo no puede responder "¿por qué el modelo respondió eso?" porque no tiene la información sobre el contexto con el que el modelo tomó esa decisión.

Respuesta 8

Describe el anti-patrón de "dashboard sin umbral" combinado con el anti-patrón de "alertas sin dueño" y la consecuente "fatiga de alertas". Cuando hay demasiadas alertas activas y algunas se ignoran sistemáticamente, el sistema de alertas deja de funcionar: cuando aparece

una alerta crítica, no recibe la atención que merece porque el equipo está acostumbrado a ignorar las alertas.

Corrección: reducir el número de alertas a las que el equipo puede atender en un día normal. Revisar cada alerta existente: ¿es Nivel 1 (logging), Nivel 2 (revisión activa) o Nivel 3 (acción inmediata)? Las de Nivel 1 deberían no generar notificaciones activas, sino solo aparecer en el dashboard operacional. Asignar un dueño explícito a cada tipo de alerta. Definir un proceso de resolución explícita: cada alerta se cierra cuando se investiga y se toma una decisión, no cuando la métrica mejora por sí sola.

Respuesta 9

La deriva del modelo ocurre cuando el comportamiento del modelo cambia sin que el sistema haya sido modificado por el equipo, típicamente porque el proveedor actualizó silenciosamente la versión del modelo de producción.

Es difícil de detectar porque no hay ningún evento en el sistema que la señale: el código no cambió, la base de conocimiento no cambió, la infraestructura no cambió. Los síntomas son cambios en el tono, la longitud, el nivel de precaución o la capacidad de seguir instrucciones complejas del model — cambios que solo son detectables al comparar el comportamiento actual con el histórico.

El mecanismo más fiable para detectarlo es el golden set ejecutado periódicamente y comparado contra la línea base. Si el mismo conjunto de casos que antes producía scores de 0.90 ahora produce consistentemente 0.82, hay evidencia de un cambio en el modelo que debe investigarse, incluyendo verificar con el proveedor si hubo una actualización.

Respuesta 10

Desafíos: con 500 solicitudes por día y un split 50/50, cada variante recibe 250 solicitudes por día. Para detectar una diferencia de 0.05 puntos en groundedness (un efecto size razonable) con 80% de potencia estadística y un nivel de significancia del 5%, se necesitan aproximadamente 1,000-1,500 solicitudes por variante, lo que requiere 4-6 semanas de experimento — un período durante el cual el sistema puede tener cambios externos (actualizaciones de la base vectorial, cambios en los patrones de consulta) que contaminan el experimento.

Recomendaciones: (1) definir antes del experimento el efecto mínimo detectable y el tamaño de muestra necesario, y comprometerse a no evaluar el experimento antes de alcanzarlo; (2) asignar usuarios, no solicitudes, a las variantes para evitar que el mismo usuario experimente

los dos sistemas; (3) monitorear las métricas secundarias (latencia, costo) además de la métrica primaria (groundedness); (4) registrar cualquier evento externo durante el experimento que pueda contaminar los resultados; (5) no concluir el experimento antes del tamaño de muestra necesario aunque los resultados intermedios parezcan significativos.

Capítulo 13 — Observabilidad, Evaluación y Optimización

Sección 15: Transición al Capítulo 14

La observabilidad da al AI Engineer la capacidad de ver qué ocurre dentro de su sistema de IA. Permite detectar cuando las respuestas son incorrectas, diagnosticar por qué lo son, medir si el sistema mejora o se deteriora con el tiempo, y operar el sistema con intención basada en datos. Es una capacidad indispensable para cualquier sistema de IA en producción.

Pero ver no es lo mismo que proteger.

El límite de la observabilidad

Todos los modos de degradación que el capítulo describió tienen en común una característica: son el resultado de dinámicas no adversariales. El modelo deriva porque el proveedor actualiza su versión. El contexto se desactualiza porque los procesos de actualización fallan. Los agentes entran en bucles porque sus criterios de terminación están mal definidos. Los usuarios producen tipos de consulta no anticipados porque sus necesidades evolucionan. Ninguno de estos problemas es el resultado de que alguien esté deliberadamente intentando dañar el sistema.

Los sistemas de IA en producción enfrentan también amenazas deliberadas: actores que intentan extraer información que el sistema no debería revelar, manipular el comportamiento del sistema a través de entradas cuidadosamente diseñadas, usar el sistema como vector para ataques contra la organización o sus usuarios, o eludir las restricciones de seguridad y cumplimiento que el sistema debe respetar.

Estas amenazas no son detectables por la observabilidad estándar. Un ataque de prompt injection —donde una entrada maliciosa intenta reescribir las instrucciones del system prompt— puede no producir ninguna alerta de latencia, costo o groundedness. El sistema responde, el contexto está presente, los scores de calidad pueden ser altos. Y sin embargo, el sistema está siendo manipulado de formas que pueden tener consecuencias graves.

Lo que la seguridad y la gobernanza agregan

El capítulo 14 desarrolla la dimensión de la operación responsable que la observabilidad por sí sola no puede proveer: cómo proteger un sistema de IA de las amenazas deliberadas que caracterizan un entorno adversarial, y cómo estructurar el gobierno de esos sistemas para que operen dentro de los límites legales, éticos y organizacionales que la organización y la regulación exigen.

La seguridad de sistemas de IA tiene un conjunto de vulnerabilidades específicas que no tienen equivalente directo en el software tradicional. Las amenazas de prompt injection, jailbreaking, data poisoning y exfiltración de información del contexto son propias de la arquitectura de los LLMs. Requieren controles específicos de diseño —no solo monitoreo de incidentes— que el AI Engineer debe incorporar en la arquitectura del sistema desde el inicio.

La gobernanza, por su parte, responde a las preguntas sobre quién tiene autoridad para tomar qué decisiones sobre el sistema: quién puede modificar el system prompt de producción, quién aprueba los cambios en la base de conocimiento, quién determina qué casos de uso son apropiados para un sistema de IA y cuáles no, cómo se documenta el razonamiento detrás de las decisiones de diseño para cumplir con los requisitos de explicabilidad que las regulaciones emergentes exigen.

La relación entre observabilidad y seguridad

Observabilidad y seguridad no son dimensiones independientes. Se refuerzan mutuamente.

La observabilidad es una condición previa para la seguridad operativa: si el sistema de IA no tiene trazabilidad de contexto, un incidente de seguridad —una respuesta que reveló información que no debía, una interacción donde el sistema fue manipulado para producir contenido inapropiado— es imposible de investigar retroactivamente. El equipo de seguridad no puede saber qué ocurrió exactamente, cómo ocurrió, y cuántas veces ocurrió antes de ser detectado.

La seguridad, a su vez, define qué es visible en la observabilidad. Si el sistema de IA maneja datos personales o información corporativa sensible, los datos que se registran en las trazas deben estar sujetos a controles de acceso, cifrado en reposo y políticas de retención que cumplan con las regulaciones aplicables. La observabilidad no puede ser ilimitada en los sistemas que manejan información sensible; debe diseñarse dentro de los límites que la seguridad y la privacidad establecen.

Lo que el capítulo 14 construye sobre este capítulo

El AI Engineer que llega al capítulo 14 con los conceptos de este capítulo internalizados tiene una ventaja: entiende que las decisiones de seguridad y gobernanza no son restricciones arbitrarias sobre el diseño técnico. Son respuestas a amenazas reales en entornos reales, y deben diseñarse con el mismo rigor que la arquitectura del contexto o el pipeline de observabilidad.

El capítulo 14 estructura esas respuestas: las vulnerabilidades específicas de los sistemas de IA, los controles de diseño que las mitigan, y el marco de gobernanza que permite a las organizaciones operar sistemas de IA dentro de los límites que la responsabilidad empresarial y la regulación exigen.

Ver qué ocurre en el sistema es el primer paso de la operación responsable. Proteger el sistema de lo que puede ocurrir es el segundo. Ambos pasos son necesarios. Ninguno es suficiente sin el otro.